Dolphin Technology



Verification using UVM

Pham Ngoc Lam

Verification Engineer

Version 25.03

September 18, 2025

Revision 2.0

© Dolphin Technology Vietnam Center. All rights reserved

Verification using UVM

Course Version 25.04

Revision 1.0

Estimate time: 6 days

© Dolphin Technology Vietnam Center. All rights reserved

Dolphin Technology

Table of Contents

1. About This Course
2. Introduction to UVM
3. Overview of DUT
4. Data Modeling
5. Simulation Phases
6. Environment and Test Classes
7. Creating an Agent
8. Configuration
9. Type Overrides and the Factory
10. Sequences
11. Connecting to a DUT
12. Multiple Agents
13. Multichannel (Virtual) Sequences
14. Building a Scoreboard
15. Transaction Level Modeling (TLM)
16. Functional Coverage Modeling
17. Register Modeling in UVM

© Dolphin Technology Vietnam Center. All rights reserved

01

About this course

Course Prerequisites
Course Objectives
Course Agenda



Dolphin Technology Vietnam Center
...to be Silicon IP Experts

© Dolphin Technology Vietnam Center. All rights reserved

Course Prerequisites

This course assumes you have the following knowledge:

- SystemVerilog language for design and verification

Knowledge of the following is useful, but not essential:

- An appreciation of object-oriented design
- Some familiarity with the Cadence Xcelium, Synopsys VCS or Siemens QuestaSim simulator

© Dolphin Technology Vietnam Center. All rights reserved

About this course

Course Objectives

The focus of this course is on features of the Universal Verification Methodology (UVM) that target complex design verification.

In this course, you will:

- Define the advantages of a standard verification methodology
- Explore the UVM Verification Component (UVC) architecture
- Control the generation of stimuli
- Create reusable UVCs
- Control environmental behaviors
- Generate stimulus sequences and check responses
- Implement a scoreboard to help check the behavior of a design
- Connect the scoreboard using Transaction-Level Modeling (TLM)
- Generate, integrate, and simulate a Register Model to help verify DUT register and memory behavior

© Dolphin Technology Vietnam Center. All rights reserved

About this course

Course Agenda

- About This Course
- Introduction to UVM
- Overview of DUT
- Data Modeling
- Simulation Phases
- Environment and Test Classes
- Creating an Agent
- Configuration
- Type Overrides and the Factory
- Sequences
- Connecting to a DUT
- Multiple Agents
- Multichannel (Virtual) Sequences
- Building a Scoreboard
- Transaction Level Modeling (TLM)
- Functional Coverage Modeling
- Register Modeling in UVM

© Dolphin Technology Vietnam Center. All rights reserved

02

Introduction to UVM

What are benefits of the UVM?



© Dolphin Technology Vietnam Center. All rights reserved

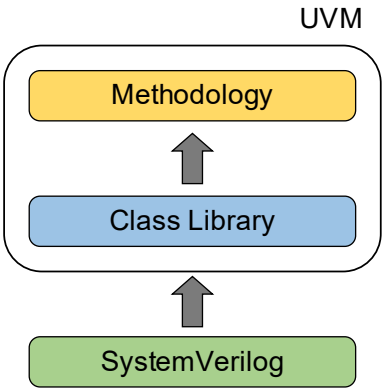
Module Objectives

In this module, you

- Discuss the benefits of the Universal Verification Methodology (UVM)
- Define the architecture of a UVM Verification Environment
- Recognize the structure of the UVM library

© Dolphin Technology Vietnam Center. All rights reserved

What is UVM?



- A class library of verification building blocks
  - Written in standard IEEE 1800 SystemVerilog
- A proven verification methodology
  - Defines how to use the class library
  - Scalable from block-level to system-level verification
- IEEE standard 1800.2
  - Documented and maintained by IEEE
  - Open source reference library from Accellera
  - Does not lock users into a single vendor solution

UVM is a library of SystemVerilog classes which implement verification building blocks along with support features such as phasing, messaging, factory etc. The library provides additional class automation, such as print, copy, comparison, cloning etc., which are not available in SystemVerilog.

The library API is defined as an IEEE standard (1800.1) and Accellera (the industry standard body) provides a reference implementation of the library. By extending our testbench environment from the UVM library, we can exploit the built-in automation and support features.

However, UVM is also a methodology for using the building blocks and features of the library. By following the methodology, you can create scalable verification components which can be used from block-level to system-level verification.

© Dolphin Technology Vietnam Center. All rights reserved

UVM Library

- Library classes provide verification functionality and building blocks.
  - Structural components:
    - Verification blocks supporting a standard architecture
  - Automation
    - Automatic implementation of print, copy, compare, etc. methods
  - Simulation control
  - Reporting
- Benefits:
  - Productivity
    - How much manual work can be saved?
  - Predictability
    - Will I meet my deadlines on time?
  - Quality
    - Will my testbench be free of bugs?

UVM is a library of SystemVerilog classes which implement verification building blocks along with support features such as phasing, messaging, etc.

Automation refers to the automatic generation of common class operations such as copy, print and compare, which are not available in SystemVerilog. By generating these for us, UVM saves us from having to write these ourselves. In general, UVM provides the building blocks for testbench generation, saving us considerable time.

Developing a testbench using UVM building blocks allows us to leverage tried and tested code and therefore minimizes bugs and errors.

© Dolphin Technology Vietnam Center. All rights reserved

Methodology

Methodology defines how to use library classes.

Benefits:

- Proven solution for verification success
- Captures best-known practices with minimal improvisation
- Provides consistency and uniformity in the way verification components are developed and used
  - Internal and commercial reuse can dramatically speed up delivery and improve quality
- Accommodates unanticipated requirements
  - Support easy configuration, customization, and modification
- Ease of use
- Supports key verification methodologies such as Metric-Driven Verification (MDV)

A good methodology, based on a standard verification component architecture, increases productivity and predictability, as it saves much time otherwise taken in planning and partitioning the testbench design.

UVM recognizes that a reusable block or third-party verification IP may never completely match the requirements of a user, and so provides powerful configuration and type-override capabilities to allow the user to customize and modify verification blocks.

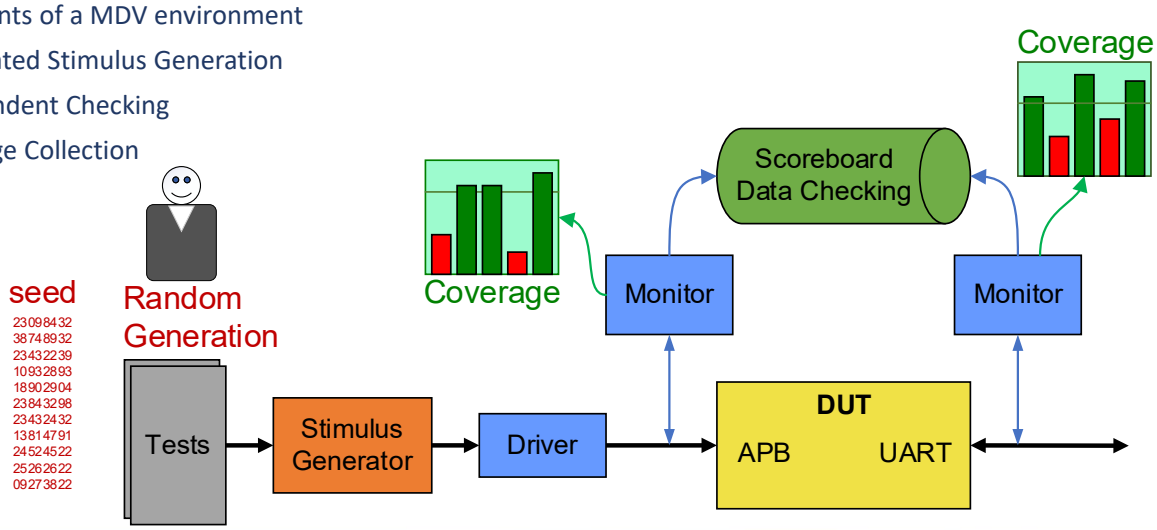
A test writer is a verification engineer who develops tests, according to the verification plan, for an existing verification environment. UVM is designed to allow a test writer to have maximum control without requiring an in-depth knowledge of the environment.

© Dolphin Technology Vietnam Center. All rights reserved

Metric-Driven Verification Review

Components of a MDV environment

- Automated Stimulus Generation
- Independent Checking
- Coverage Collection



A Metric-Driven Verification methodology is composed of three main features:

- Random stimulus to test the widest possible design functionality with the smallest stimulus code.
- Assertions and independent checking to define behavior which should or should not happen, and so find bugs in the design.
- Coverage so we can determine what has and has not been tested. Coverage feeds back to the test writer, who can then modify the stimulus to close coverage holes.

© Dolphin Technology Vietnam Center. All rights reserved

Creating Reusable Environments

Having a standard architecture for verification components is key for sharing code

- Easier to learn and use components
- Easier for joint debug

Some of the requirements of component architecture include the following:

- Quick adoption and readability
- Modular and well-structured topology
- Seamless support for both system-level and block-level
- Conform to industry best practices

Việc có một kiến trúc chung (standard architecture) cho các thành phần xác minh giúp dễ dàng **tái sử dụng mã nguồn** (code sharing) giữa các nhóm hoặc dự án. Điều này tiết kiệm thời gian, giảm lỗi và tăng hiệu quả.

- **Dễ học và sử dụng**: Một kiến trúc thống nhất giúp các kỹ sư (dù mới hay có kinh nghiệm) nhanh chóng làm quen và sử dụng các thành phần mà không cần học lại từ đầu.
- **Hỗ trợ gỡ lỗi chung (joint debug)**: Khi tất cả các thành phần đều tuân theo một chuẩn, các nhóm có thể cùng nhau phân tích và sửa lỗi dễ dàng hơn, vì mọi người đều hiểu cấu trúc.

Yêu cầu của kiến trúc thành phần xác minh:

- **Dễ tiếp cận và dễ đọc (Quick adoption and readability)**: Mã phải được viết rõ ràng, dễ hiểu để người mới có thể sử dụng ngay.
- **Cấu trúc mô-đun và tổ chức tốt (Modular and well-structured topology)**: Các thành phần nên được chia thành các mô-đun độc lập, dễ bảo trì và mở rộng.
- **Hỗ trợ cả cấp hệ thống và cấp khối (Seamless support for system-level and block-level)**: Thành phần xác minh phải hoạt động tốt trong cả môi trường kiểm tra lớn (hệ thống) và nhỏ (khối riêng lẻ).
- **Tuân thủ các tiêu chuẩn công nghiệp (Conform to industry best practices)**: Ví dụ, tuân theo các phương pháp như UVM (Universal Verification Methodology), một tiêu chuẩn phổ biến trong xác minh phần cứng.

© Dolphin Technology Vietnam Center. All rights reserved

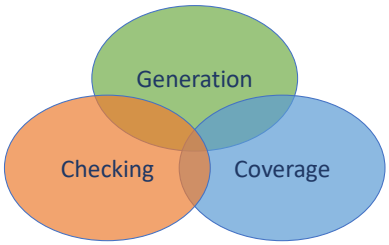
UVM Verification Component (UVC)

Definition

- Verification IP block (VIP)
- Reusable, pre-verified, configurable, plug-and-play
  - Architected and packaged in a consistent way
- Complete: provides all the logic required for verification
  - Needs to be re-used and configured as a whole

Two types of UVC

- Interface UVCs
  - Bus-based UVCs (for example, PCI and AHB)
  - Communication UVCs (for example, Ethernet, MAC, Datalink)
- Module UVCs (for example, UART core, ALU module)
  - Scoreboard
  - Register model



© Dolphin Technology Vietnam Center. All rights reserved

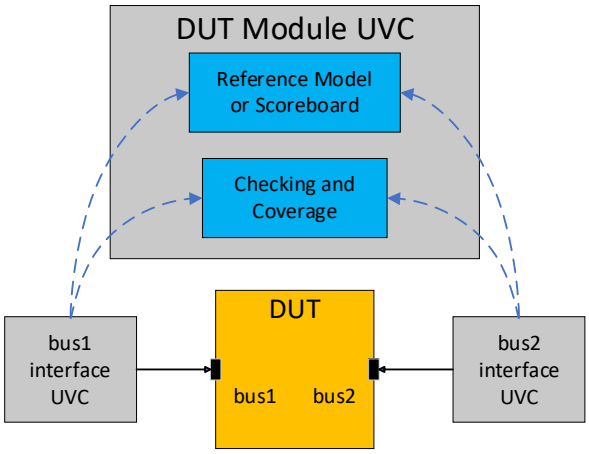
Module and Interface UVCs

Interface UVC

- Specific to a protocol, bus, or set of signals on the DUT
- Generates stimulus for the interface
- Reusable everywhere interface is used
- Set architecture and configuration

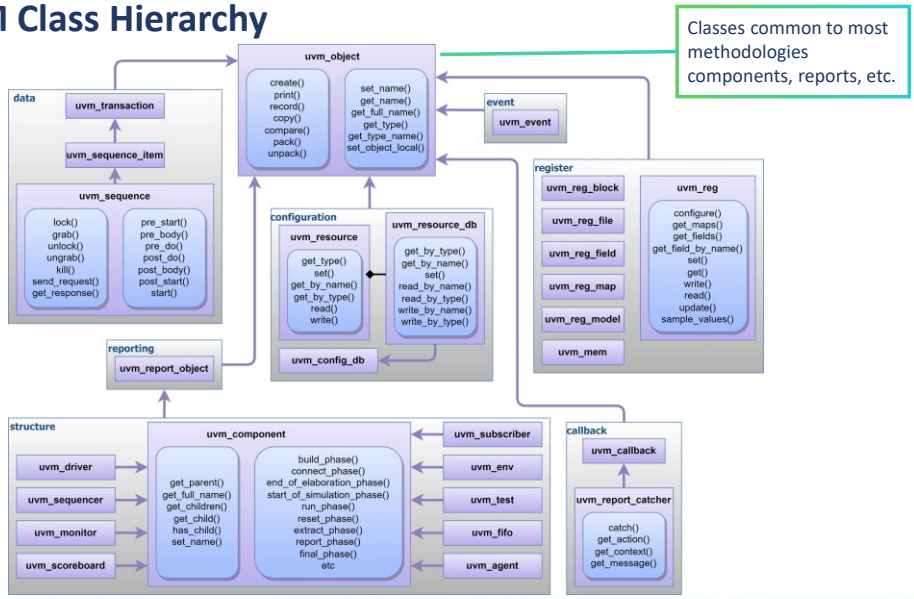
Module UVC

- Specific to DUT or flavor of DUT
- Provides checks and coverage across interface UVCs
- Reusable everywhere DUT is used
- No set architecture



© Dolphin Technology Vietnam Center. All rights reserved

Simplified UVM Class Hierarchy



Although the UVM class library looks complicated (and this diagram is a deliberate simplification), most UVM verification environments are written by extending the “leaf” classes only, e.g., uvm_env, uvm_test, uvm_monitor, etc.

There are three main layers to the UVM library:

- A SystemVerilog implementation of the OSCI TLM standard. This defines how class components can communicate with each other.
- The structural elements layer, which provide the features common to most verification methodologies, such as component construction, messaging, and phasing.
- The methodology layer that provides classes to specifically support the methodology and architecture for creating UVM VIP

Thư viện UVM nhìn phức tạp nhưng được đơn giản hóa khi sử dụng

- Thư viện UVM bao gồm nhiều lớp (classes) với các mối quan hệ phức tạp, khiến nó có vẻ khó tiếp cận ban đầu. Sơ đồ trên đơn giản hóa cấu trúc UVM để dễ hiểu hơn.
- Khi xây dựng môi trường xác minh, người dùng thường chỉ cần mở rộng (extend) các lớp "lá" (leaf classes).

Ba layers chính của thư viện UVM:

- TLM layer (Transaction-Level Modeling): Tầng này định nghĩa cách các components giao tiếp với nhau.

© Dolphin Technology Vietnam Center. All rights reserved

▪ Structural elements layer: Tầng này cung cấp các tính năng chung cho hầu hết các phương pháp kiểm thử, là nền tảng để xây dựng môi trường kiểm thử có tổ chức, dễ mở rộng và bảo trì, ví dụ:

▪ Hỗ trợ tạo và kết nối các components trong một cấu trúc phân cấp.

▪ Cung cấp cơ chế ghi log, báo lỗi, hoặc thông báo trạng thái (uvm_info, uvm_error, etc).

▪ Quản lý các giai đoạn của quá trình kiểm thử (build, connect, run, cleanup, report).

▪ Methodology layer: Tầng này cung cấp các lớp hỗ trợ đặc biệt phương pháp và kiến trúc cho việc xây dựng UVM VIP.

© Dolphin Technology Vietnam Center. All rights reserved

03

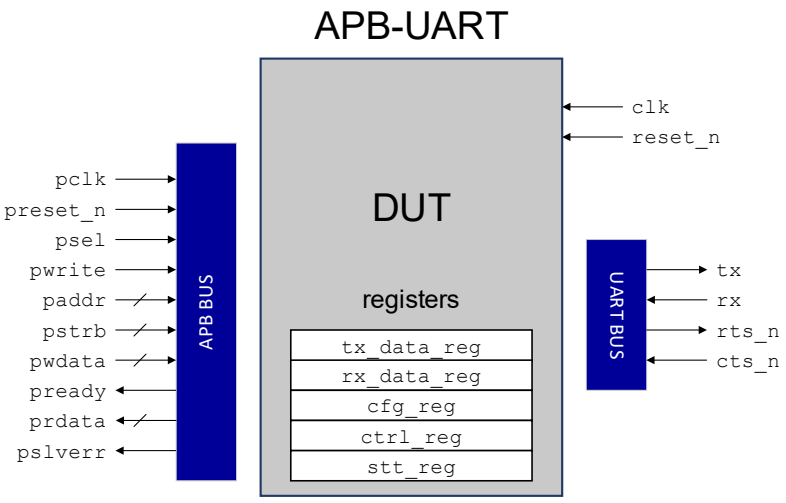
Overview of DUT and Project

Define the UVM verification environment for testing the APB UART

© Dolphin Technology Vietnam Center. All rights reserved

Overview of DUT and Project

DUT: APB UART

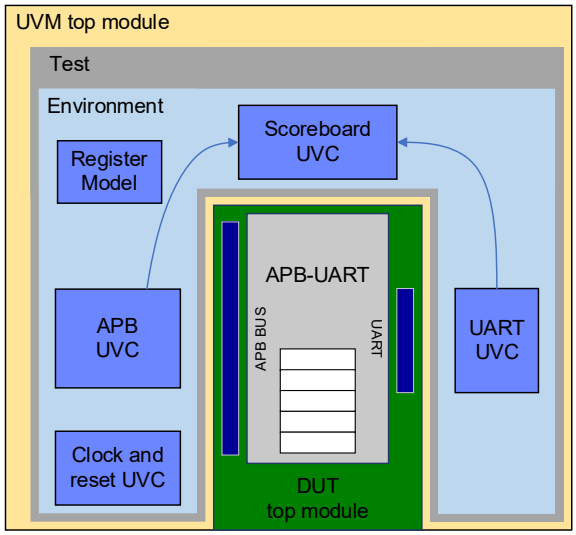


We will be using the APB UART as the DUT for the example throughout this course.

© Dolphin Technology Vietnam Center. All rights reserved

Overview of DUT and Project

How Will We Verify the APB UART



- Create a UVC for each interface and clock/reset.
- Environment instantiates and configures UVCs.
- Test instantiates testbench and controls stimulus.
- Add a mechanism to coordinate stimulus from multiple UVCs.
- Add scoreboard for end-to-end checking.
- Add a Register Model to help verify DUT register behavior.
- DUT top module instantiates DUT and interfaces.
- UVM top module instantiates test and connects UVC interfaces.

We will be using the APB UART as the DUT for the example throughout this course.

© Dolphin Technology Vietnam Center. All rights reserved

04

Data Modeling

Model transaction data items for the inputs and outputs of the DUT



© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Model transaction data items for the inputs and outputs of the DUT
- Define default constraints and switches to control randomization
- Add the necessary constructs to use data items in a UVM environment
- Add UVM automation
  - Using the UVM “opt-in” system to allow printing, copying, comparing, etc.

© Dolphin Technology Vietnam Center. All rights reserved

Data Item Example: APB Transaction

```
class apb_item;
  rand bit [12:0] paddr;
  rand bit [31:0] pdata;
  rand bit        pwrite;
  rand bit [ 3:0] pstrb;

  // control fields
  rand int        item_delay;

  // status fields
  rand bit        rw_error;

endclass : apb_item
```

Fields to be randomized

Clock cycle delay between packets

Status transfers

© Dolphin Technology Vietnam Center. All rights reserved

Basic Constraints

```
class apb_item;
  rand bit [12:0] paddr;
  rand bit [31:0] pdata;
  rand bit        pwrite;
  rand bit [ 3:0] pstrb;

  rand int        item_delay;
  rand bit        rw_error;

  // Default Constraints
  constraint read_strb {
    !pwrite -> pstrb == '0;
    solve pwrite before pstrb;
  }

endclass : apb_item
```

Dependency constraint

Change probability distribution of `pwrite` and `pstrb`

Dolphin Confidential Information

The APB specification defines that PSTRB must not be active during a read transfer.

© Dolphin Technology Vietnam Center. All rights reserved

Class Methods

```
class apb_item;
  rand bit [12:0] paddr;
  rand bit [31:0] pdata;
  rand bit        pwrite;
  rand bit [ 3:0] pstrb;
...
endclass : apb_item
```

What kind of operations may be required for such a data item?

- Randomization
- Printing
- Comparing
- Copying
- Packing
- Waveform dumping

Randomization is defined in SystemVerilog

UVM class library provides the rest

Dolphin Confidential Information

There are a standard set of operations that you would want to carry out on a class. As a minimum, you would want to randomize a class instance and print the property values. You would also want to compare different instances of the same class and copy class properties from one instance to another. You may want to pack all (or selected) class properties into a single vector or data structure for transmission to the DUT. You may also wish to modify the representation of an instance in a waveform viewer.

SystemVerilog provides very little infrastructure for classes – only the randomization method is built-in to the language. Normally the user would have to create custom implementations of the required methods, but UVM can provide these for us.

Standard UVM class operations are:

- Copy – copies a field from one instance to another. Also used in clone.
- Compare – compares a field from one instance with another.
- Print – prints the field to the standard output. Also sprint which creates the print string, but does not send it to the output.
- Pack/unpack – concatenates the fields into a single integral array.
- Record – enables vendor-specific transaction recording in a waveform viewer.

© Dolphin Technology Vietnam Center. All rights reserved

Data Modeling with UVM

```
class apb_item extends uvm_sequence_item;
  rand bit [12:0] paddr;
  rand bit [31:0] pdata;
  // other field declarations
...
  function new (string name = "apb_item");
    super.new(name);
  endfunction

  `uvm_object_utils_begin(apb_item)
    `uvm_field_int(paddr,UVM_ALL_ON)
    `uvm_field_int(pdata,UVM_ALL_ON)
    // more field automation
  `uvm_object_utils_end
endclass : apb_item

apb_item p_item = new("p_item");
```

Inherit from `uvm_sequence_item`

Data constructor with `string name` argument and default value

Must call `super.new(name);`

Must have utility macro block

Argument is name of class

Enable automation for `apb_item` fields

Instance name should match handle name

Dolphin Confidential Information

To enable a data class for UVM, we have to add 3 elements as a minimum:

1. Extend from the UVM subclass `uvm_sequence_item` to inherit UVM functionality.

2. Add a UVM data constructor with a single argument `name` of type `string` with a default value (by convention we use the data class name). This is the instance name of the class. The constructor must call `super.new(name)` as the first line to pass the instance name argument up into the `uvm_sequence_item` class.

Failure to use a default value for name gives the following compilation error:

- `*E,FAABP1 task, function, or assertion instance does not specify all required formal arguments`

3. Add a field automation block using `uvm_object_utils` macros.

`` `uvm_object_utils `` macros are used to enable common operations declared in `uvm_object` (`copy`, `compare`, `print`, etc.)

© Dolphin Technology Vietnam Center. All rights reserved

UVM Field Macros

Use field macros to enable automation methods for class properties

- `copy`, `compare`, `pack`, `record`, `print`

Systax: `` `uvm_field_* ``(<field_name>, <flags>)

- <field_name> must be an existing property identifier
- <flags> define what is supported for that field

Non-array types use these macros:

```
`uvm_field_int( <field_name>, <flags> )                  // integral types
`uvm_field_object( <field_name>, <flags> )               // class handle
`uvm_field_string( <field_name>, <flags> )               // strings
`uvm_field_event( <field_name>, <flags> )                // events
`uvm_field_real( <field_name>, <flags> )                 // reals
`uvm_field_enum( <enum_type>, <field_name>, <flags> )    // enums
```

Macros for enumeration types have an additional argument for the type name

Dolphin Confidential Information

You usually have a field macro for every field (property) of the class. These can only be declared inside the object utilities block (between `` `uvm_object_utils_begin `` and `` `uvm_object_util_end ``).

If you do not declare a field macro for a property, then the property is excluded from all automation methods, i.e., it will not be printed, copied, compared, recorded or packed.

The name of the macro depends on the type of the property. The macro has two or more arguments. The first argument is the name of the field. The field type must be compatible with the macro name. The second argument is a flag list which defines which automation methods are enabled for the field.

The macros shown here are for properties of integral, class, string, event and enumerate types. Remember an integral type in Verilog is any packed one-dimensional array of `bit` or `logic`, including `integer` and `int` types.

Field macros for enumerate fields have an additional, initial macro which must be the name of the enumerate type, implying that a `typedef` should be used to name enumerate types for class properties. Fields declared of an antonymous enumerate type can not be automated.

© Dolphin Technology Vietnam Center. All rights reserved

Array Type Field Macros

unpacked 1-D dynamic arrays

```
`uvm_field_array_int()
`uvm_field_array_object()
`uvm_field_array_string()
`uvm_field_array_enum()
```

unpacked 1-D static arrays

```
`uvm_field_sarray_int()
`uvm_field_sarray_object()
`uvm_field_sarray_string()
`uvm_field_sarray_enum()
```

- Array macros depend on the array type

queues

```
`uvm_field_queue_int()
`uvm_field_queue_object()
`uvm_field_queue_string()
`uvm_field_queue_enum()
```

associative arrays

```
`uvm_field_aa_<d_type>_<ix_type>()
`uvm_field_aa_int_string()
`uvm_field_aa_object_string()
`uvm_field_aa_string_string()
`uvm_field_aa_int_int()
`uvm_field_aa_int_integer()
`uvm_field_aa_int_int_unsigned()
`uvm_field_aa_string_int()
`uvm_field_aa_object_int()
```

Dolphin Confidential Information

Simple logic and bit arrays use the `uvm_field_int macro on the previous page. These array macros are for unpacked one-dimensional static arrays, one-dimensional dynamic arrays and associative arrays.

For a dynamic array, the choice of macro depends on the data type of the array, so for:

```
bit [7:0] payload[];
```

The field automation macro is:

```
`uvm_field_array_int(payload, <flags>)
```

For a unpacked static array, the choice of macro depends on the data type of the array, so for:

```
string strarr [7:0];
```

The field automation macro is:

- `uvm_field_sarray_string(strarr, <flags>)

For associative arrays, d_type is the data type and ix_type is the index type. So for:

```
logic [7:0] assoc [string];
```

The field automation macro is:

```
`uvm_field_aa_int_string(assoc, <flags>)
```

There are many other associative array macros beyond those listed here.

© Dolphin Technology Vietnam Center. All rights reserved

Flag Arguments for Field Macros

Flags are numeric and can be combined using a bitwise OR '|' or AND '+' operator.

Operation Flag	Description
UVM_ALL_ON UVM_DEFAULT	All field operations enabled (copy, compare, print, record, and pack) – default setting.
UVM_NOCOPY	Excludes the field for copy and clone.
UVM_NOCOMPARE	Excludes the field for compare.
UVM_NOPRINT	Excludes the field in print and sprint.
UVM_NOPACK	Excludes the field in pack and unpack.
UVM_NORECORD	Excludes the field in vendor-specific transaction recording.

Additional Print/Record Radix Flags for Integral Types				
UVM_BIN	UVM_DEC	UVM_UNSIGNED	UVM_OCT	UVM_HEX
Binary %b	Signed decimal %d	Unsigned decimal %u	Octal %o	Hex %h (default)

Dolphin Confidential Information

Field macro flag arguments define which class operations are implemented for the specified field.

UVM_ALL_ON or UVM_DEFAULT enable all operations. The UVM_NO* flag options only exclude specific operations but implicitly allow everything else.

Standard UVM class operations are:

- Copy – copies a field from one instance to another. Also used in clone.
- Compare – compares a field from one instance with another.
- Print – prints the field to the standard output. Also used in sprint which creates the print string, but does not send it to the output.
- Pack/unpack – concatenates the fields into a single integral array and back.
- Record – enables vendor-specific transaction recording in a waveform viewer.

Flags are implemented as a vector with separate bits for every operation. Therefore flags can be combined with | and + operations.

Additional print flags to control how integral fields are displayed in print or record. The default is UVM_HEX. UVM_DEC displays values as signed (2's complement) whereas UVM_UNSIGNED displays them as unsigned (binary).

The print flags are for scalar or arrayed integral fields only. Field macros of non-integral types, such as uvm_field_string, sets the print option automatically to the correct type. Setting a print flag in a non-integral field macro has no effect.

© Dolphin Technology Vietnam Center. All rights reserved

Field Macro Example

Every data item property should have a field automation macro.

```
typedef enum bit {GOOD_PARITY, BAD_PARITY} parity_t;
class apb_item extends uvm_sequence_item;
  ...
  rand bit [31:0] pdata[];
  rand int        item_delay;
       bit        parity;
  rand parity_t   parity_type;

  // field declaration automation flags
  `uvm_object_utils_begin(apb_item)
    ...
    `uvm_field_array_int(pdata, UVM_ALL_ON)
    `uvm_field_int(item_delay, UVM_ALL_ON + UVM_NOCOMPARE)
    `uvm_field_int(parity, UVM_ALL_ON + UVM_BIN)
    `uvm_field_enum(parity_t, parity_type, UVM_ALL_ON)
  `uvm_object_utils_end

endclass : apb_item
```

Enables automation for `ahb_item` fields

`UVM_NOCOMPARE` overrides `UVM_COMPARE` set by `UVM_ALL_ON`

Print radix

Dolphin Confidential Information

Usually everything is enabled and print radix options are added:

e.g., enable everything and print in unsigned binary:

- `` `uvm_field_int(data, UVM_ALL_ON + UVM_UNSIGNED); ``

To disable specific operations, it is usually more readable to turn everything on (with either `UVM_ALL_ON` or `UVM_DEFAULT`), and then add exceptions and print radix options.

e.g., enable everything except compare and print in binary:

```
`uvm_field_int(data, UVM_ALL_ON + UVM_NOCOMPARE + UVM_BIN);
```

However flag options to disable specific operations implicitly allow all other operations, therefore the above could be written as|:

- `` `uvm_field_int(data, UVM_NOCOMPARE + UVM_BIN); ``

Multiple disable flag options must be added separately:

e.g., disable copy and compare only and print in Hex (default)

```
`uvm_field_int(data, UVM_ALL_ON + UVM_NOCOPY + UVM_NOCOMPARE);
`uvm_field_int(data, UVM_NOCOPY + UVM_NOCOMPARE);
```

© Dolphin Technology Vietnam Center. All rights reserved

Copy and Clone Automation Methods

Copy method

- `function void copy(uvm_object rhs);`
- Copies all fields from rhs argument to the calling object
  - Except instance name

Clone method

- `function uvm_object clone();`
- Creates a new instance and copies all fields
  - Including instance name
- As clone returns `uvm_object`, you will need to use `$cast`

```
apb_item p1, p2, p3;          // data item handles
int ok;

initial begin
  p1 = new("p1");             // create p1 instance
  p2 = new("p2");             // create p2 instance
  ok = p1.randomize();
  p2.copy(p1);                // copy p1 to p2
  $cast(p3, p1.clone());      // create instance p3 and copy
end                           //  all fields from p1 to p3
```



What is the difference between copy and clone?

Dolphin Confidential Information

`copy` Syntax

```
function void copy(uvm_object rhs);
```

UVM also defines a function do_copy which, if defined in a data class, will be called before the copy operation. This allows custom copying for fields, e.g., conditional copying of one field depending on the value of another. In such an example, the customized field must be excluded from standard copying using field flags (`UVM_NOCOPY`).

`clone` Syntax

```
function uvm_object clone();
```

Question: What is the difference between `clone()` and `copy()`?

Answer: `clone()` creates a new object and then copies all the fields across (clone = construct + copy)

Another important difference between `copy` and `clone` is in the handling of the instance name for the object. When copying from `p1` to `p2`, the instance name is not copied. Therefore `p2` keeps its original instance name `p2`. When cloning `p1` to `p3`, `p3` is given the instance name from `p1`. There is a UVM method to update the instance name:

```
$cast(p3, p1.clone());
p3.set_name("p3");
```

© Dolphin Technology Vietnam Center. All rights reserved

Compare Automation Method

Compare method

- `function void compare(uvm_object rhs, uvm_comparer comparer=null);`
- Compares the contents of the `rhs` object with the calling object
- Takes an optional `comparer` argument (described in the reference manual)
- Returns a 1 if all compared fields match, otherwise returns 0, and prints report

```
apb_item p1, p2;
int ok;

initial begin
  p1 = new("p1");
  p2 = new("p2");
  ok = p1.randomize();
  ok = p2.randomize();
  if(!p2.compare(p1))
    ...
end
```

Compare p1 and p2

Where would you see the compare method?
How can we exclude fields from comparison?

Report for miscompare (see later for details)

```
UVM_INFO @ 0: reporter [MISCMP] Miscompare for p2.paddr: lhs = 'h2 : rhs = 'h3
UVM_INFO @ 0: reporter [MISCMP] 3 Miscompare(s) (1 shown) for object p1@2614 vs. p2@2578
```

Dolphin Confidential Information

`compare` Syntax

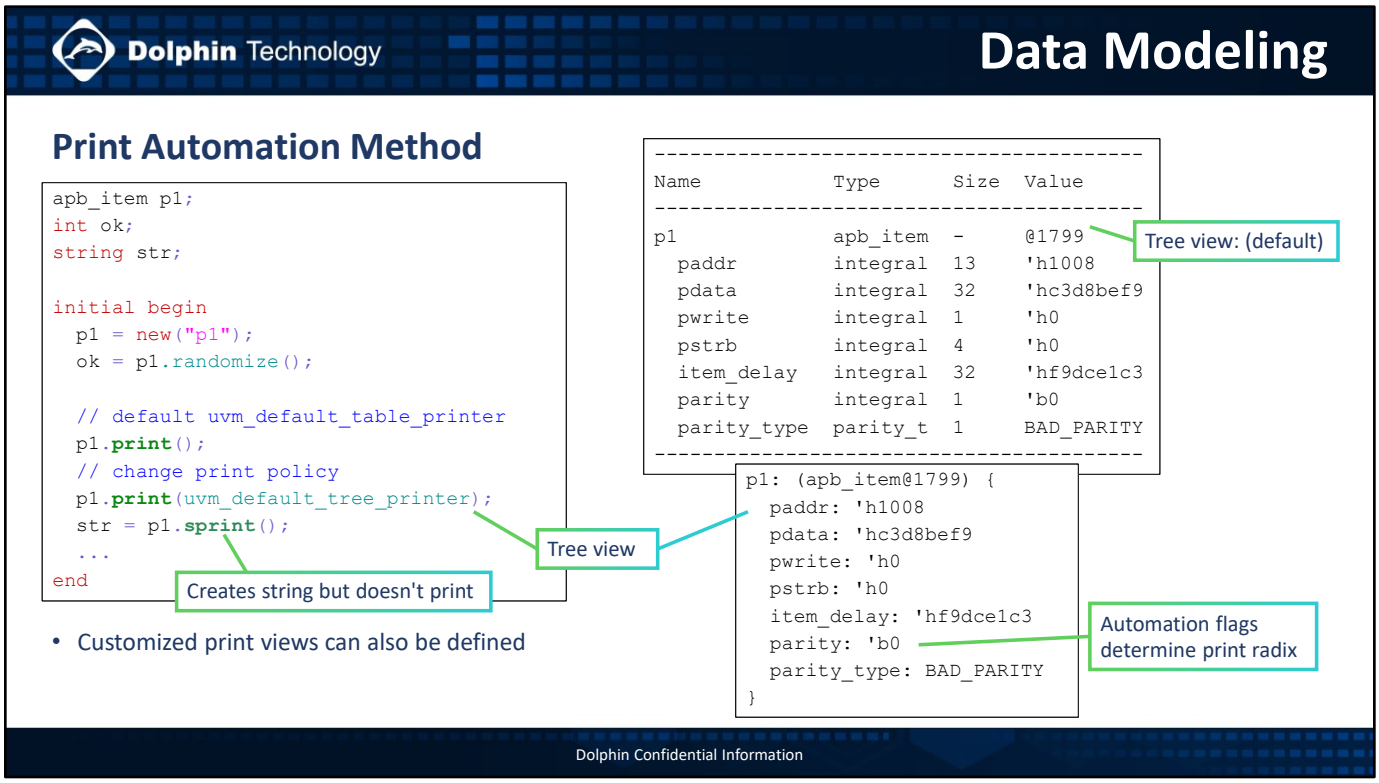
`function bit compare(uvm_object rhs, uvm_comparer comparer=null);`

Note that by default, only the first property mismatch is shown, and miscompares are info report (see later for report severity). This default behavior can be changed via a third argument to the `compare` of type `uvm_comparer`.

`uvm_comparer` is a built in UVM class containing settings for default comparison. By creating an instance of this class and updating the settings we can modify the default comparison behavior.

`uvm_comparer` also defines a function `do_compare` which, if defined in a data class, will be called before the compare operation. This allows custom comparison for the object fields, e.g., conditional comparison of one field depending on the value of another. In such an example, the customized field must be excluded from standard comparison using field flags (`UVM_NOCOMPARE`).

© Dolphin Technology Vietnam Center. All rights reserved



The `print()` method converts the packet values into a string, and writes the string to the simulator output. There is also a sub-method called `sprint()`. This creates the print string but does not write it to the output. We use `sprint()` in UVM messaging (see later).

Syntax

```
function void print ( uvm_printer printer = null );
  function string sprint ( uvm_printer printer = null );
```

`uvm_printer` is a built in UVM policy class containing settings and formatting for default printing. By extending this class and modifying the settings we can redefine the default print operation to create new custom print policies.

Three print policies are pre-defined:

- `uvm_default_table_printer (default)`
- `uvm_default_line_printer`
- `uvm_default_tree_printer`

UVM also defines a function `do_print` which, if defined in a data class, will be called before the print operation. This allows custom printing for object fields, e.g., conditional printing of one field depending on the value of another. In such an example, the customized field must be excluded from standard printing using field flags (`UVM_NOPRINT`).

© Dolphin Technology Vietnam Center. All rights reserved

Creating a UVM Package

Most user-defined UVM classes are placed in separate files.

Classes can only be compiled from a module or package scope.

Recommendation:

- Include related class files into a package
- Import the package into modules where required

The UVM library is supplied in the package `uvm_pkg`.

- Import package to access the library.
- The macro file `uvm_macros.svh` must be included separately.

```
class apb_item extends uvm_sequence_item;
  ...
endclass
```

apb_item.sv

```
package apb_pkg;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "apb_item.sv"
   ...
endpackage
```

apb_pkg.sv

Dolphin Confidential Information

Two declarations are required to access the UVM library.

First, you need to import the UVM package

```
import uvm_pkg::*;
```

Second, in order to resolve the macros used in the UVM library, which are expanded as a precompilation step, you directly include the UVM macros definition file:

```
`include "uvm_macros.svh"
```

For maintenance and portability, it is a convention that most UVM classes are declared in separate files. However, the class files cannot be compiled directly, but can only be compiled from a module or package scope.

The recommendation is to include related UVM class files into a package, and import the package into the module (or package) where it is required. Remember that the package file must be separately compiled on the command line.

© Dolphin Technology Vietnam Center. All rights reserved

05

Simulation Phases

Describe the UVM simulation phase implementation



Dolphin Technology Vietnam Center

...to be Silicon IP Experts

© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Discuss the need for simulation phases
- Describe the UVM simulation phase implementation
- Create hierarchy using build phases

© Dolphin Technology Vietnam Center. All rights reserved

Simulation Phases

Simulation Phases for Components

Design verification is divided into phases:

Testbench creation, configuration, run, check, report results, etc.

Most verification components perform set tasks in each simulation phase:

- Build and connect hierarchy during the "creation" phase.
- Stimulus generation during the "run" phase.

Some components only need to perform tasks in specific phases.

- Scoreboard examining results in the "check" phase.

Phases must run in order.

- Next phase cannot begin until the current phase is complete
  - For example, don't check results until the run is complete.

Having a set of standard pre-defined phases allows easy integration of multiple components.

Dolphin Confidential Information

Verification of a design in simulation can be divided up into a series phases. For example:

- Creating and connecting the testbench components
- Configuring the testbench and design
- Running stimulus
- Checking scoreboards and reporting results

For some components, their main functionality is executed in specific phases, for example, a scoreboard examining results in the check. Other components may have actions in every phase.

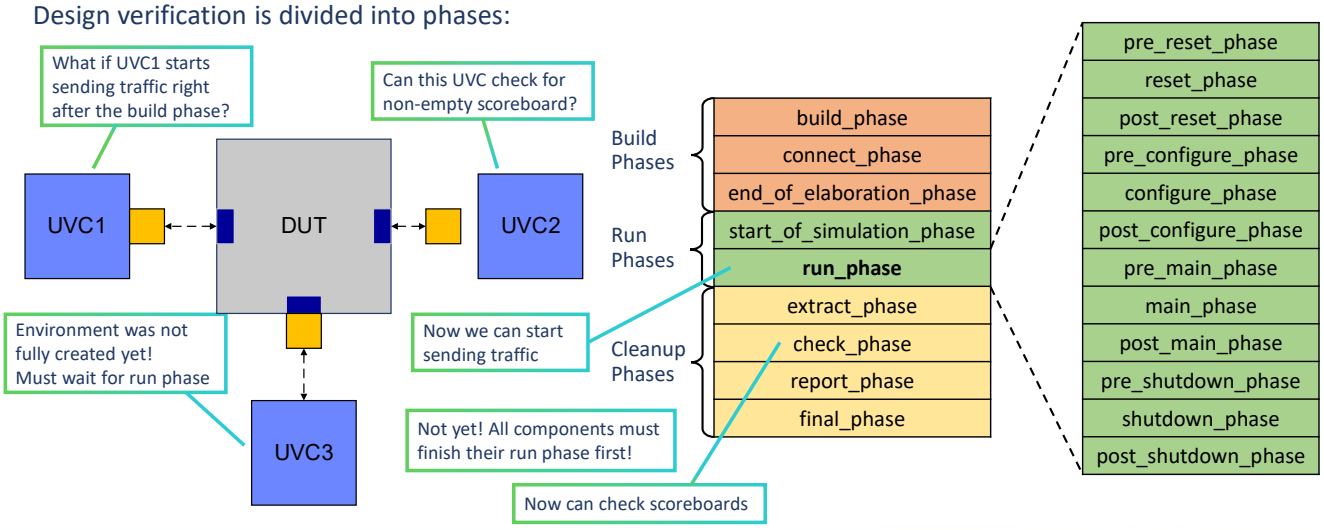
For clean simulation, the phases must run in order, and the next phase should not begin until the current phase is completely finished. For example, the scoreboard cannot check results until every UVC has finished sending stimulus.

UVM defines a set of standard phases for easy integration of multiple testbench components.

© Dolphin Technology Vietnam Center. All rights reserved

Simulation Phases and Coordination

Design verification is divided into phases:



Dolphin Confidential Information

Verification of a design in simulation can be divided up into a series phases. For example:

- Creating and connecting the testbench components
- Configuring the testbench and design
- Running stimulus
- Checking scoreboards and reporting results

For some components, their main functionality is executed in specific phases, for example, a scoreboard examining results in the check. Other components may have actions in every phase.

For clean simulation, the phases must run in order, and the next phase should not begin until the current phase is completely finished. For example, the scoreboard cannot check results until every UVC has finished sending stimulus.

UVM defines a set of standard phases for easy integration of multiple testbench components.

© Dolphin Technology Vietnam Center. All rights reserved

Simulation Phases

UVM Simulation Phase Names and Descriptions

All phases except `run_phase()` execute in zero time as functions

build_phase	Build top-level testbench topology
connect_phase	Connect environment topology
end_of_elaboration_phase	Post-elaboration activity (e.g., print topology)
start_of_simulation_phase	Prepare for simulation
run_phase	Task – run-time execution of the test
extract_phase	Gathers details on the final DUT state
check_phase	Processes and checks the simulation results
report_phase	Simulation results analysis and reporting
final_phase	Tie up loose ends, close files

- User-defined phases can also be added and there are additional run-time phases

Dolphin Confidential Information

These are the names of the pre-defined phases in UVM.

`build_phase` – for building the testbench component hierarchy.

`connect_phase` – for making connections between components in the hierarchy.

`end_of_elaboration_phase`, `start_of_simulation_phase` – these two phases are for debug (printing the component hierarchy); initial configuration and any other pre-run activity.

`run_phase` – simulation of the design.

`extract_phase`, `check_phase`, `report_phase` – these phases are for extracting and checking results, and generating reports on the simulation status and success.

`final_phase` – any other activity before end of simulation.

All phases execute in zero time, and are implemented using functions, except `run_phase`, which can consume time and must be implemented as a task.

You can also add user-defined phases, although this can break reusability. There are also additional subphases of the run phase – called runtime phases. These are described on the next slide.

© Dolphin Technology Vietnam Center. All rights reserved

Simulation Phases

Phase Method Reference

All phases automatically called by the simulator

```
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  ...
function void connect_phase(uvm_phase phase);
  ...
function void end_of_elaboration_phase(uvm_phase phase);
  ...
function void start_of_simulation_phase(uvm_phase phase);
  ...
task run_phase(uvm_phase phase);
  ...
function void extract_phase(uvm_phase phase);
  ...
function void check_phase(uvm_phase phase);
  ...
function void report_phase(uvm_phase phase);
  ...
```

Build phase must contain super.build_phase(phase);

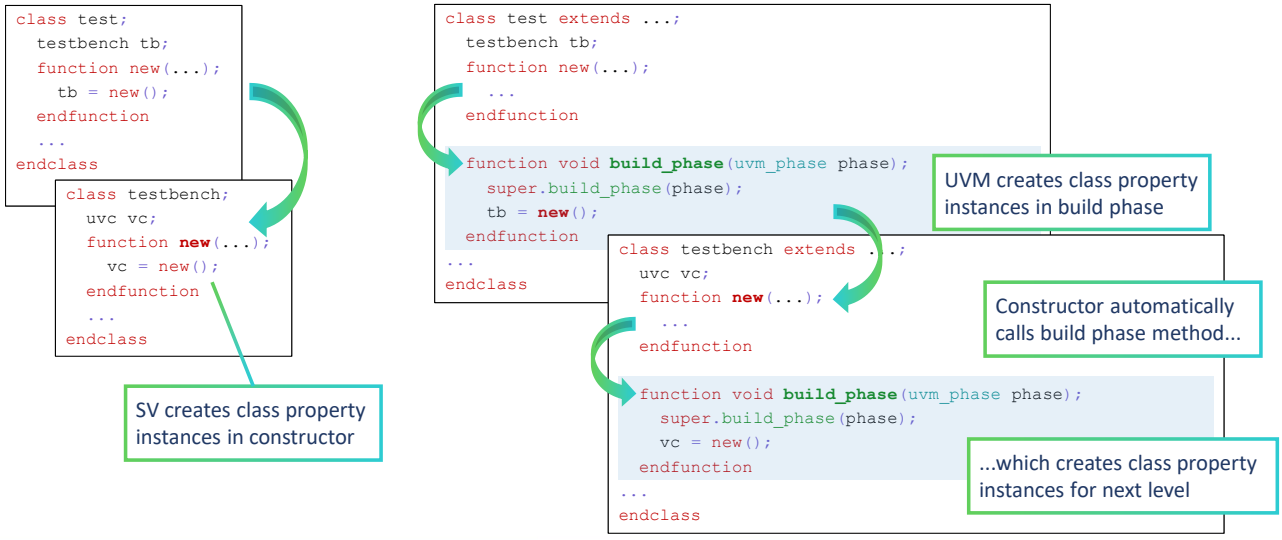
No other phase needs contain a super call

Run phase is a task,all other phase methods are functions

Add these methods to your components to execute code in the required phase.

© Dolphin Technology Vietnam Center. All rights reserved

Build Phases for Aggregate Classes



Dolphin Confidential Information

The component hierarchy for a UVM verification environment is created using properties which are instances of other classes (i.e., using aggregate or composite classes).

In SystemVerilog, we create class property instances in the constructor. So the constructor of the `test` class contains a call to the constructor of the `testbench` class to create the `tb` component instance. Likewise the `testbench` constructor creates the next level of hierarchy by calling the `uvc` constructor to create the `vc` instance.

However in UVM we use the build phase to create the component instances. So the build phase of `test` contains a call to the `testbench` constructor to create the `tb` instance. In UVM, executing the testbench constructor automatically calls the testbench build phase (if it is defined) to create the next level of hierarchy – the `vc` instance of class `uvc`.

The build phase is a design pattern – a standard solution to a common object-oriented design problem. We'll see why we use build phases in the configuration module later.

© Dolphin Technology Vietnam Center. All rights reserved

How Are Phases Activated?

Execute the global task run_test() from a module scope to start phasing.

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  ...
initial begin
  run_test();
  ...
```

Execution:

- Build phase functions execute top-down
- Run phase tasks execute in parallel
- All other phases execute bottom up
- All components finish the previous phase before the next phase starts

Timing:

- Run phase and all the run-time phases are tasks and can consume time
- All other phases are defined with functions and execute in zero time

To use any phase, define the phase method, and it will be executed during that phase.

© Dolphin Technology Vietnam Center. All rights reserved

06

Environment and Test classes



Dolphin Technology Vietnam Center

...to be Silicon IP Experts

© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Create and use environment classes
- Define the requirement and implementation of UVM test classes
- Select specific test classes to execute
- Debug hierarchies using `print_topology()`
- Use UVM reports for debug and error messaging

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

Building the APB-UART Verification Environment

In this module, we add the environment and test class layers.

Test are the top level of the UVM environment:

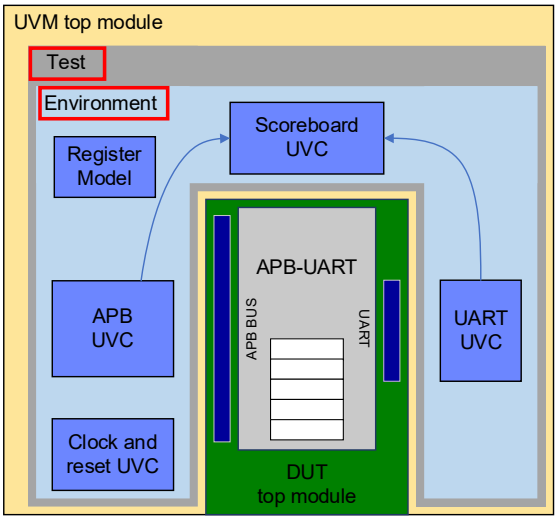
- All UVM hierarchy is under a test class

Test classes:

- Control the stimulus generated by UVCs
- Instantiate the environment class

The environment:

- Instantiates and configures UVCs



Dolphin Confidential Information

The topmost component in a UVM environment is the test class. Its key roles are:

- Instantiation of the testbench component
- Configuration of UVC properties and policies
- Selection of stimulus for interface UVCs
- Controlling generation of data stimulus using type overrides

The testbench:

- Instantiates UVCs (both interface and module)
- Instantiates other components such as multichannel sequencers, register models etc.
- Configures the UVCs

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

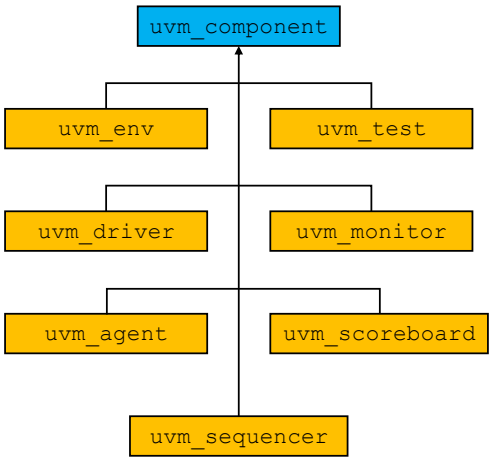
Components and Objects

Data items:

- Define stimulus
- Extend from uvm_sequence_item
- Have **object** utility macros and constructors

Components:

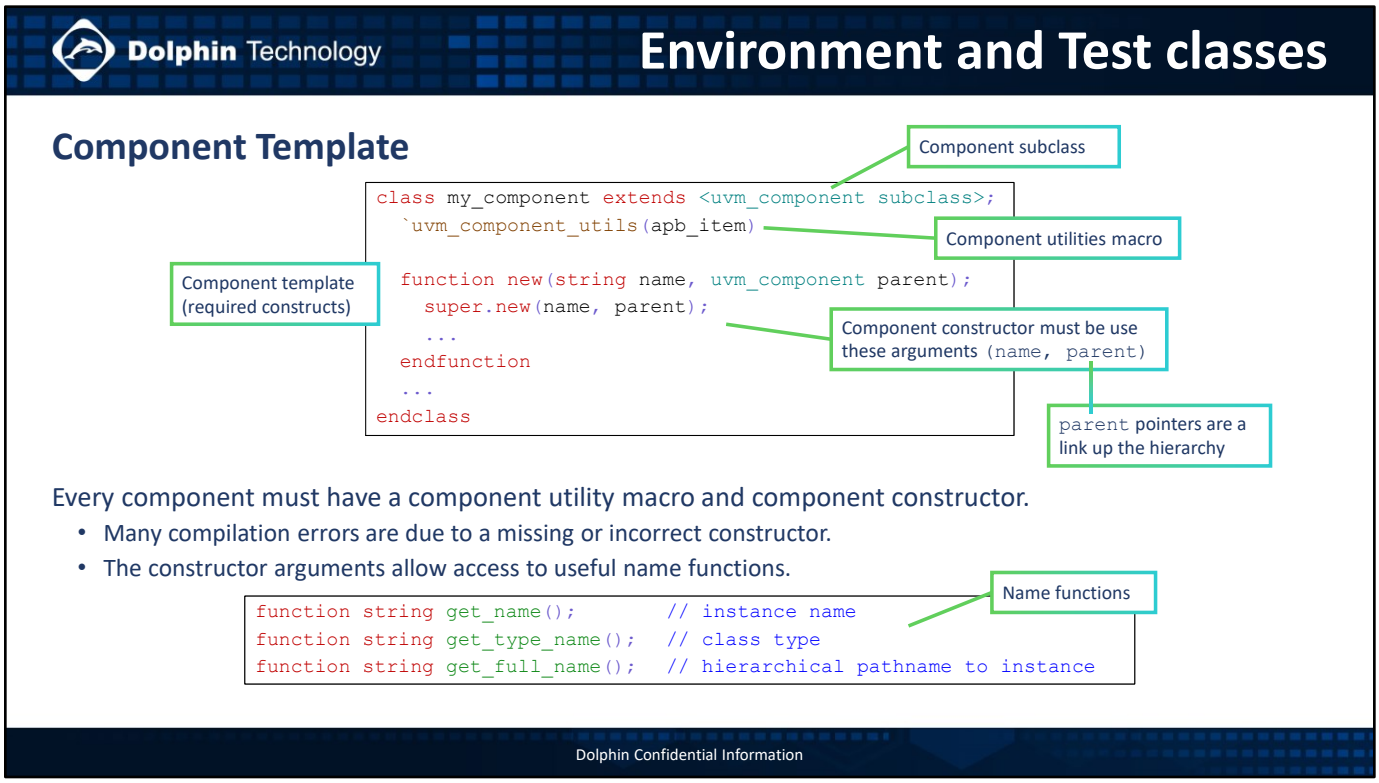
- Form the verification environment using an aggregate class hierarchy
- Are derived from uvm_component
- UVM provides sub-classes for your environment, test, UVC, agent, sequencer, etc.
- Have **component** utility macros and constructors



Dolphin Confidential Information

We have already seen how UVM provides a parent class for data items (`uvm_sequence_item`) as a means of accessing the UVM functionality for data.
UVM also defines parent classes for components. `uvm_component` is the root class for all component classes and we could extend all our user-defined components directly from this class. However, UVM defines specific subclasses of `uvm_component` for every UVC subcomponent, for example `uvm_sequencer`, `uvm_driver`, `uvm_monitor`, `uvm_agent`. These component-specific subclasses define essential functionality for the component implementations and should be used in preference to `uvm_component`.

© Dolphin Technology Vietnam Center. All rights reserved



Just like a data class, component classes need a UVM utility macro and a specific constructor. However component classes have a different macros and constructors than data classes.

Every component needs a *component utility macro* to enable automation for units. The utility macro comes in two forms:

- `` `uvm_component_utils_begin(<type_name>) ``
- `// field macros`
- `` `uvm_component_utils_end ``

or

- `` `uvm_component_utils((<type_name>) ``

If your component class declares local data properties then we use the begin/end utility which allows us to apply field automation to the data properties.

If your component does not declare local data properties, we can use the second macro form.

Every component class requires *a component constructor* with an instance name argument of type string and a parent pointer argument of type uvm_component. The constructor must call `super.new(name,parent)` as the first line to pass the arguments up into the UVM library.

The constructor arguments allow several useful methods, for example `get_full_name()` which returns the hierarchical pathname for the component as a string.

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

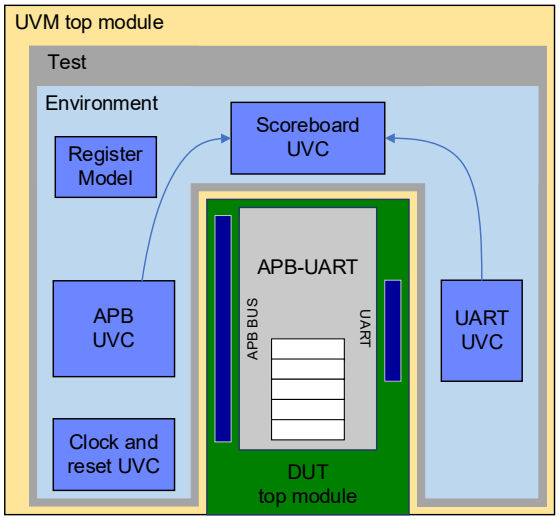
What is an Environment

An environment encapsulates all verification components for a specific DUT

Instantiates and configures:

- Interface UVCs:
  - APB, UART
- Module UVCs:
  - Scoreboard
  - Reference model
- Other components:
  - Register model

Could have multiple environments for one DUT



Dolphin Confidential Information

The testbench must carry out the following:

- Declare handles for all UVCs.
- Configure the UVCs in the build_phase method.

Configuration of the UVCs is vitally important. Complex UVCs may require many configuration statements for correct operation.

Typical UVC configuration tasks include:

- Setting the default_sequence property of a UVC sequencer.
- Enabling transaction recording by globally setting recording_detail properties.
- Setting UVCs to be passive (if required).
- Defining the number and type of agents in a UVC, e.g., transmit or receive.

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

Environment Shell

apb_uart_env.sv

```
class apb_uart_env extends uvm_env;
  `uvm_component_utils(apb_uart_env)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // configure and create uvcs
    ...
  endfunction
  ...
endclass
```

No testbench subclass (could use uvm_component)

Component utilities macro

Component constructor

UVC instances will be added later

- UVC instances will be added to the testbench later

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

Test Requirements

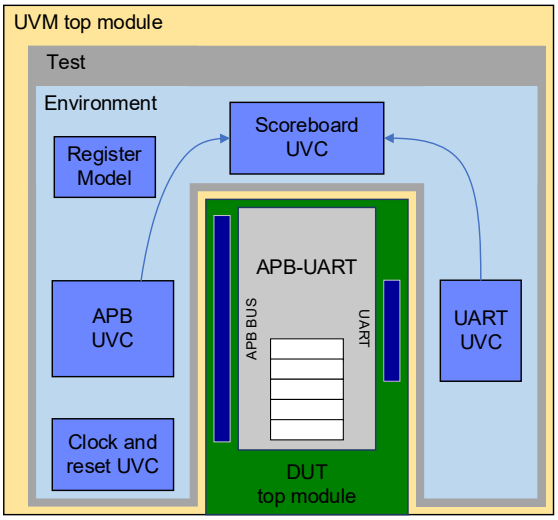
Test classes control UVC stimulus

There are usually multiple test classes:

- Constrained random tests tend to be much shorter
- Many test classes are declared in the same file
  - Test library
- Allow easier test reuse
  - Use inheritance to avoid describing hierarchy and settings for every test

We want to:

- Reduce knowledge of the environment for writing tests
  - Provide a standard test-writer interface
  - Minimal knowledge and effort in creating or enhancing tests
  - Ability to write hierarchy-independent tests
- Reduce compilation time when launching a specific test
  - Compile multiple tests at the same time



Dolphin Confidential Information

There are usually many test classes for each DUT, applying different stimulus or configurations to achieve different verification goals. In UVM, it only takes a few lines of code to set stimulus or configuration, therefore each test class is quite small. For this reason, and contrary to what we have seen so far, we declare many test component classes in a single file. The file is called a test library.

A test component must meet the following requirements:

Compilation efficiency. There are typically many different test components in one or more test library files. We don't want to recompile the entire testbench and DUT when switching from one test to another. Therefore we need to be able to compile all tests at the same time, and then easily switch between different tests using the same simulation kernel.

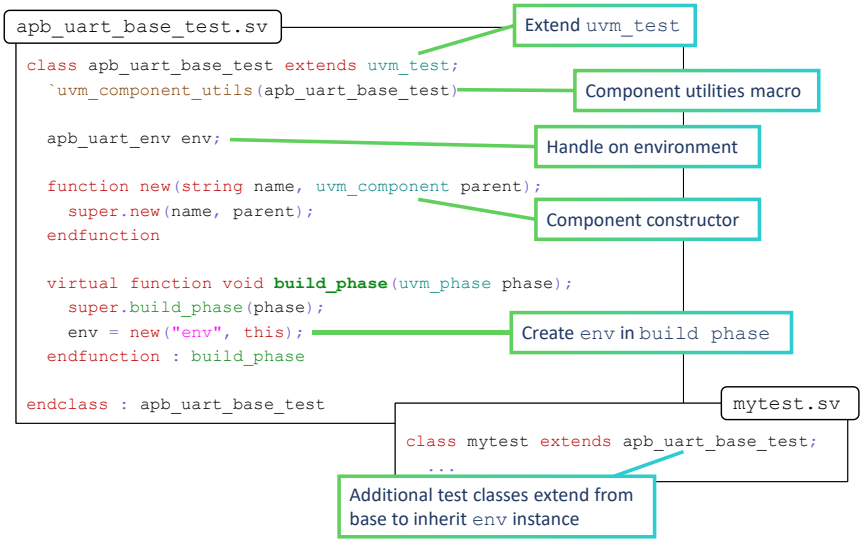
Reuse. The instantiation of the next level of the component hierarchy can be declared in a user-defined "base" test class, which is then extended by test subclasses. The subclasses can define individual stimulus and configuration without the need to declare the component hierarchy.

The test class provides a test writer interface for defining verification of the DUT. This allows test writers to define stimulus and configuration to achieve specific verification goals, with minimal knowledge of the verification hierarchy or design protocols, and with minimal effort.

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

Simple Test Example



Dolphin Confidential Information

The test class contains:

- A component utilities macro
- A component constructor
- An instance of the testbench component created in a `build_phase()` method

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

Selecting a Test

top.sv — Option 1

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  import apb_pkg::*;
  import uart_pkg::*;

  `include "apb_uart_env.sv"
  `include "apb_uart_base_test.sv"
  `include "mytest.sv"
  ...
  initial begin
    run_test("mytest");
  end
endmodule
```

- Import APB UVC
- Import UART UVC
- Include environment
- Include base test
- Include a test class
- Test class is explicitly named
  This test is *always* run during simulation

top.sv — Option 2

```
module top();
  import uvm_pkg::*;
  `include "uvm_macros.svh"

  import apb_pkg::*;
  import uart_pkg::*;

  `include "apb_uart_env.sv"
  `include "apb_uart_base_test.sv"
  `include "mytest.sv"
  ...
  initial begin
    run_test("apb_uart_base_test");
  end
endmodule
```

- Use command-line option to select a test

```
%xrun ... +UVM_TESTNAME=mytest ...
```

+UVM_TESTNAME takes precedence over the run_test() argument

Dolphin Confidential Information

We can select which test class to execute by passing the test type name as a string argument to the run_test method. If the test class cannot be found or is undefined, a fatal runtime error is issued.

Passing the test class name as an option to the command line is a much more flexible method to select a test. This allows us to create multiple test classes, compile them all, and select a specific test from the command line each time the simulation is run.

Changing the test class does not require recompilation, as long as the chosen test class has already been compiled.

© Dolphin Technology Vietnam Center. All rights reserved

Environment and Test classes

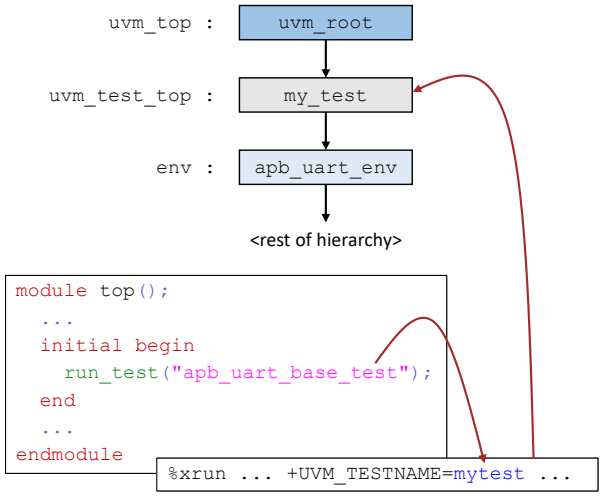
UVM Hierarchy

The topmost component is an instance of uvm_root named uvm_top.

- Top level for phasing, searching, configuration, debugging, etc.
- Gives access to useful methods:
  - run_test()
  - print_topology()

What does run_test do?

- Creates an instance of the defined test in the handle uvm_test_top.
  - With a parent of null
- Initiates simulation phases which build the rest of the test hierarchy.



Dolphin Confidential Information

The topmost component is an instance of `uvm_root` named `uvm_top`, which is created automatically. `uvm_top` is a singleton design pattern – i.e., only one instance of `uvm_root` can be created. The instance is globally visible throughout the UVM environment, and so is useful for any search, configuration and debug calls which need to be applied from the top of the hierarchy.

The `uvm_top` handle allows access to various useful methods, for example `run_test` is a globally visible method of uvm_root.

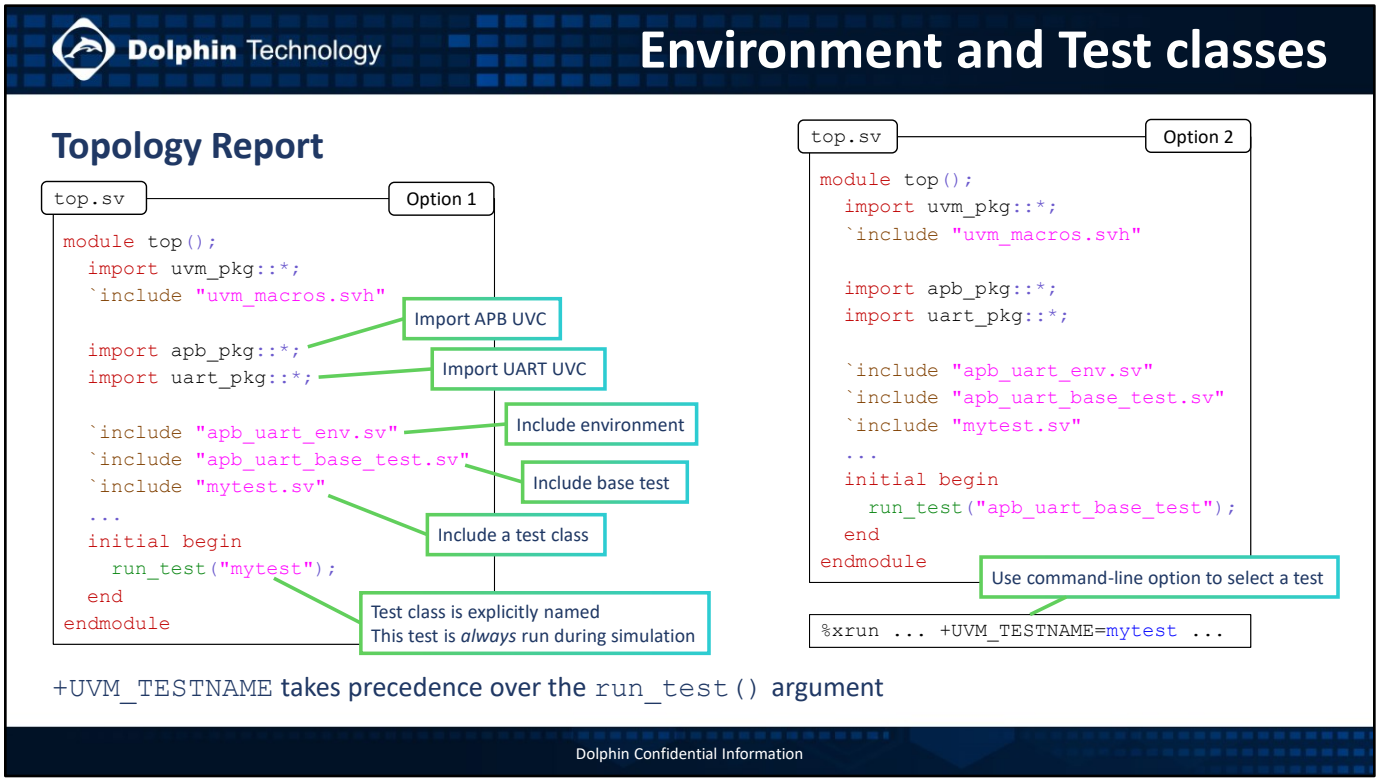
The `run_test` call:

1. Declares a handle of the selected test component named `uvm_test_top`.
2. Creates the test instance with a parent of `null`.
3. Initiates the phasing.

The phasing will execute the `build_phase` method of the test component, which then creates the rest of the verification hierarchy.

The `uvm_top` handle also allows access to `print_topology` which prints the entire component hierarchy. This is very useful for debugging hierarchy issues. It should be invoked once all the hierarchy has been built and connected, e.g., in the `end_of_elaboration` phase.

© Dolphin Technology Vietnam Center. All rights reserved



We can select which test class to execute by passing the test type name as a string argument to the run_test method. If the test class cannot be found or is undefined, a fatal runtime error is issued.

Passing the test class name as an option to the command line is a much more flexible method to select a test. This allows us to create multiple test classes, compile them all, and select a specific test from the command line each time the simulation is run.

Changing the test class does not require recompilation, as long as the chosen test class has already been compiled.

© Dolphin Technology Vietnam Center. All rights reserved

Reports for Debug and Error Message

Debugging activity inside a large environment with many UVCs is critical.

Need to report:

- Errors
- Debug
- Progress

Reports need to be categorized via severity:

- Fatal, Error, Warning, Info

Need to link actions with reports

- Exit simulation on fatal
- Stop simulation after a set number of errors

Need a different mechanism than simulator messages to avoid filtering effects.

Dolphin Confidential Information

Once we have one of more UVCs inside a large verification environment, we need to able to report what is happening during simulation.

We want to display:

- Errors, both in simulation results and in creating the verification environment.
- Debug messages to help understand when things go wrong.
- Progress reports to show the simulation is doing what it should.

Some messages (e.g., errors) will be more important than others (e.g., progress reports). Therefore we need to be able to categorize messages by severity:

- Fatal – the simulation has failed and cannot continue.
- Error – we have a problem, but simulation can continue.
- Warning – there is a possible problem or anticipated issue.
- Info – for progress reports.

Severity allows us to link specific actions with messages, for example to stop the simulation on a fatal message or after a set number of error messages. A summary of the messages of each severity at the end of the simulation will enable us to understand the simulation results at a glance.

We need a standalone messaging mechanism for UVM to allow separate control of simulation and simulator reports and to ensure portability.

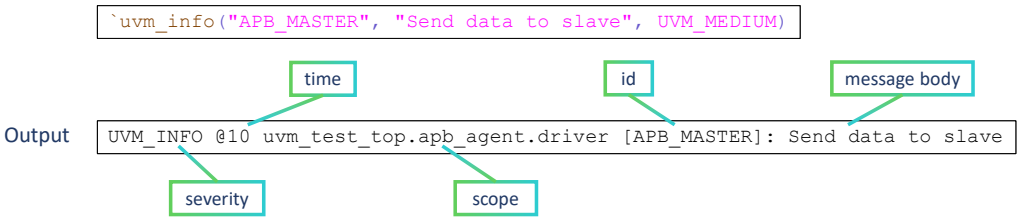
© Dolphin Technology Vietnam Center. All rights reserved

Report Macros: Info

Syntax:

```
`uvm_info(string id, string message, int verbosity)
```

- Other macros for severity `` `uvm_fatal ``, `` `uvm_error ``, `` `uvm_warning ``



Tips:

- Could use `get_type_name()` to give an id of the class type
- Use `$sformatf()` to create message string using `$display` syntax

Dolphin Confidential Information

The macro is a call to a built-in report method. The macros are more efficient than calling the method directly.

Full Method Syntax

```
function void uvm_report_* (string id, string message,
        int verbosity_level=0, string filename="", int line=0);
```

Where

- `*` is one of `fatal`, `error`, `warning` or `info`.
- `id` is an identifier for the message. You can configure report output and actions using the ID. Use a `get_type_name()` call to set id argument to the current class type name.
- `message` is the output text. It must be a string, so `$display` cannot be used directly. The `$sformatf()` system function creates a string using `$display` syntax.

The following arguments are optional:

- `verbosity` is an integer which allows further filtering of messages.
- `filename` and `line` can be used to specify the location of the message by name of file and line number of message

The report macros add the `filename` and `line` arguments automatically.

The fatal, error and warning report macros also set the `verbosity` argument to ensure these messages are always displayed.

© Dolphin Technology Vietnam Center. All rights reserved

Verbosity

```
`uvm_info(get_type_name(), $sformatf("APB item:\n%s", item.sprint()), UVM_LOW)
```

Like print but only creates string

Verbosity is a predefined int value that can be used to filter reports.

- `UVM_NONE`, `UVM_LOW`, `UVM_MEDIUM` (default), `UVM_HIGH`, `UVM_FULL`, `UVM_DEBUG`

By changing the verbosity maximum, reports can be filtered.

- Reports with a verbosity **greater than** the maximum are **not** processed

Several ways to change verbosity, for example:

- Simulator arguments
  - Global `+UVM_VERBOSITY=verbosity`
  - Hierarchical `+uvm_set_verbosity=comp,id,verbosity,phase`

Dolphin Confidential Information

Reports with a verbosity *greater* than the maximum are ignored. For example, if the verbosity is set to `UVM_LOW`, all reports coded with `UVM_MEDIUM` and higher will not print.

© Dolphin Technology Vietnam Center. All rights reserved

Full Range of Report Macros

```
`uvm_info(string id, string message, int verbosity)
`uvm_warning(string id, string message)
`uvm_error(string id, string message)
`uvm_fatal(string id, string message)
```

- `` `uvm_info() `` conditionally checks the verbosity level before performing any string manipulation to increase performance
- The verbosity of warning, error, and fatal are set to UVM_NONE so these reports cannot be hidden by the verbosity setting
- Fatal reports end simulation by default
- A count of occurrences of each report type is printed at the end of simulation
- Report behavior can be customized using a report catcher

Dolphin Confidential Information

There are four UVM report macros, one for each severity. The macros call UVM report functions. However using the macro call is more efficient, as this checks the verbosity level before creating the string message. If the verbosity setting prevents the report from displaying, the message is not created.

Report macros for warning, fatal and error severities do not have verbosity arguments. The verbosity of these reports is automatically set to UVM_NONE so that they will always display, regardless of the verbosity threshold.

By default, only a Fatal severity will stop the simulation, although this behavior can be changed (see next slide).

At the end of simulation, a report summary is printed reporting a count of the reports displayed by severity and also by ID.

By default, the output of a UVM report is standardized. The format and order and amount of information can be customized by adding a report catcher to the report generator. Report catchers can also modify the severity of a message if, for example, different UVCs are not using severity consistently.

© Dolphin Technology Vietnam Center. All rights reserved

Report Actions

Default actions are based on severity.

- Actions can be changed for severity, id, or hierarchy

Severity	Default Action
uvm_info	UVM_DISPLAY
uvm_warning	UVM_DISPLAY
uvm_error	UVM_DISPLAY + UVM_COUNT
uvm_fatal	UVM_DISPLAY + UVM_EXIT

Actions	Description
UVM_NO_ACTION	Do nothing
UVM_DISPLAY	Send report to standard output
UVM_LOG	Write report to file
UVM_COUNT	Increment quit counter*
UVM_EXIT	Terminate simulation
UVM_CALL_HOOK	Execute hook methods
UVM_STOP	Execute $stop to enter simulation interactive mode

*A maximum number of UVM_COUNT actions can be set, after which the simulation finishes:

- Default max_quit_count is 0, indicating no maximum

```
xrun ... +UVM_MAX_QUIT_COUNT=3 ...
```

Dolphin Confidential Information

UVM defines default actions for reports by severity. For example, `uvm_fatal displays the report and exits the simulator.

UVM provides commands to hierarchically change default actions by severity or report id. So we can override the default actions for a report in a specific hierarchical path, with a specific ID and of a specific severity.

The UVM_COUNT action increments a counter each time a report with the action is raised. A maximum value for the count can be defined, after which the simulation exits. Default value is 0 indicating no maximum value. Easiest and most flexible way to set the maximum is with the command line option +UVM_MAX_QUIT_COUNT, allowing the maximum to be changed for every simulation run.

The UVM_LOG action writes the report to a file. By default, no file is specified, even if UVM_LOG action is defined, although a report with the UVM_DISPLAY action will still be written to the simulator transcript window and hence to a simulation log file.

© Dolphin Technology Vietnam Center. All rights reserved

07

Creating an Interface UVC



Dolphin Technology Vietnam Center

...to be Silicon IP Experts

© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

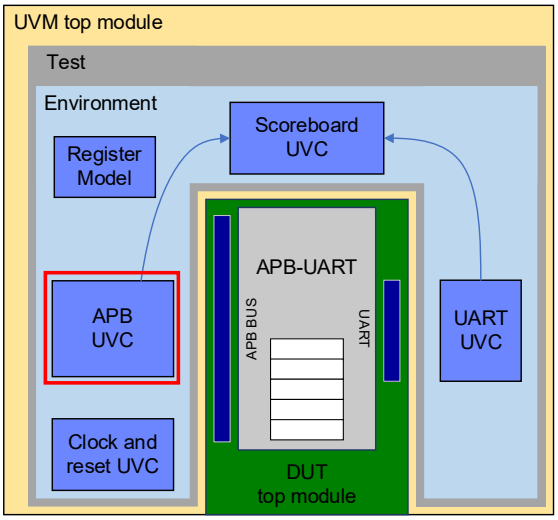
In this module, you

- Create the basics of a simple interface UVC
- Define the recommended directory structure for UVC files

© Dolphin Technology Vietnam Center. All rights reserved

Building the APB-UART Verification Environment

The APB UVC will drive APB transactions into the DUT



© Dolphin Technology Vietnam Center. All rights reserved

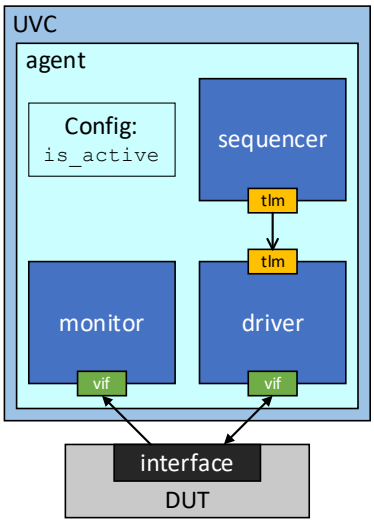
Creating an Interface UVC

Interface UVC Hierarchy Review

UVC
- Top level
- Contains one or more agent instances

Agent
- Contains instances of the sequencer, driver and monitor
- Configurable – is_active flag indicates whether the agent is active or passive
- If agent is active – driver, sequencer and monitor are constructed
- If agent is passive – only the monitor is constructed

Monitor
- Collects information from the DUT
- Connects to the DUT via a virtual interface



Sequencer
- Controls generation of the stimulus
- Points to list of data items – sequence
- Connects to the driver via a TLM interface
- Generates next sequence item on driver request

Driver
- Connects to the sequencer via a TLM interface
- Gets the next data item from the sequencer
- Drives data on the DUT interface with the correct timing
- Indicates to the sequencer that the item is done
- Connects to the DUT via a SystemVerilog virtual interface

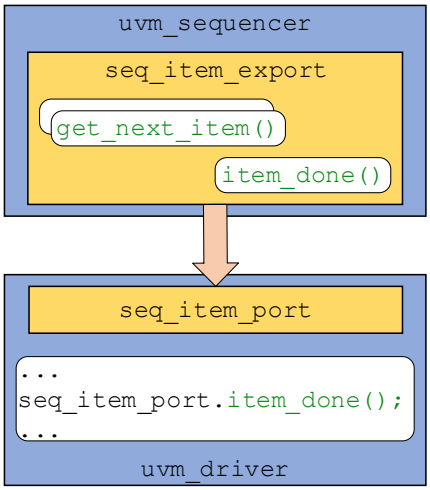
Dolphin Confidential Information

This is a reminder of the structure and functionality of an interface UVC.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

Sequencer-Driver TLM Connection



uvm_sequencer contains a built-in TLM connection – **seq_item_export**.

- This defines communication methods:
  - `get_next_item()`, `item_done()` etc.

uvm_driver contains a built-in TLM connection – **seq_item_port**

The driver **seq_item_port** is connected to the sequencer **seq_item_export**.

The driver can then execute the sequencer communication methods.

- This allows the driver to "pull-down" data from the sequencer.

By extending from **uvm_sequencer** and **uvm_driver**, we can use this connection.

Dolphin Confidential Information

UVM provides a built-in connection for sequencer and driver, defined as part of the `uvm_sequencer` and `uvm_driver` classes.

`uvm_driver` has a port connection object `seq_item_port`. `uvm_sequencer` has an export connection object `seq_item_export`. These two objects must be explicitly connected together by the agent of the UVC.

The sequencer connection object defines a number of communication methods to handle the synchronization and transfer of a data item (e.g., `apb_master_item`) between driver and sequencer.

The user simply has to call predefined communication methods off the driver connection object.

`seq_item_*` is a specialized TLM connection. TLM allows communication between components and comes in many different forms.

© Dolphin Technology Vietnam Center. All rights reserved

Sequencer-Driver Operation

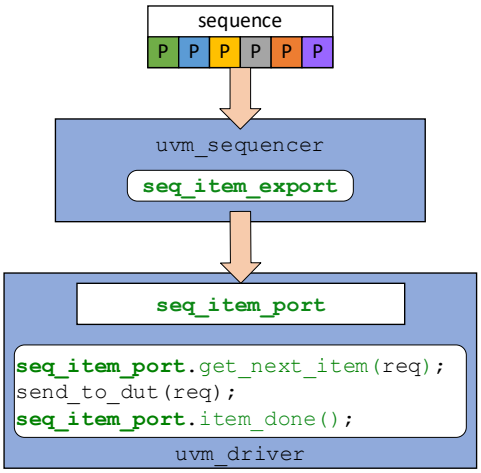
Sequencer is linked to a sequence

- A pattern of individual data items
  - More details later in class

Sequencer-driver operation

- Driver calls `get_next_item()`
- Sequencer generates next data item from sequence and sends to driver
  - Returned as output argument to `get_next_item()`
- Driver sends data item to DUT by driving interface signals
- Driver indicates the item is finished by calling `item_done()` from port

Operation continues until all sequence items sent



Dolphin Confidential Information

A sequencer is linked to a sequence which defines stimulus in terms of a pattern of individual data items. The sequencer will feed these items, one at a time, to the driver on demand.

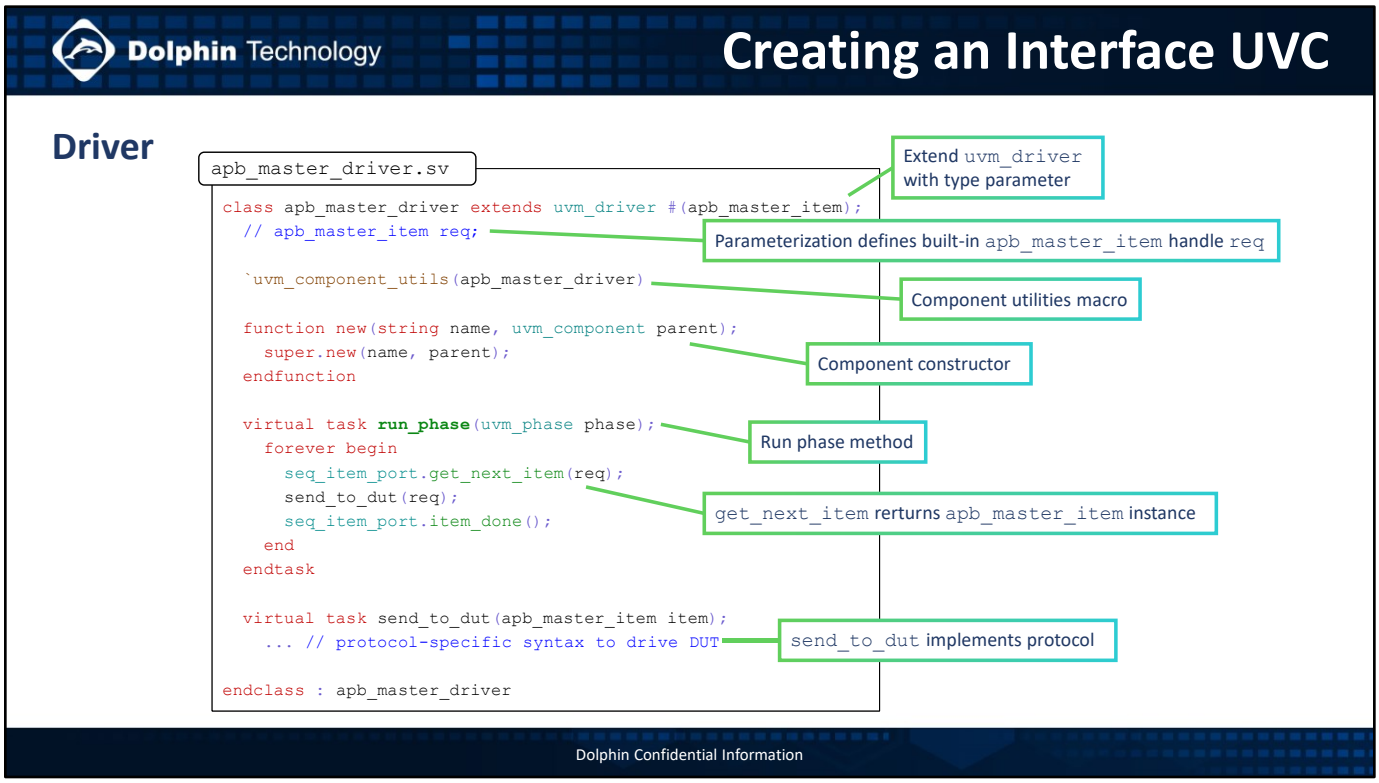
The driver calls the communication methods of the sequencer from its `seq_item_port` to pull-down the individual data items and send them to the DUT.

The most common procedure is as follows:

- Driver requests an item by calling `get_next_item()` from the `seq_item_port`. The simulator executes the method in the sequencer to which the driver is connected. The sequencer finds the next data item from the sequence; creates and generates the item (usually through randomization) and return a pointer to the item as the return value of the `get_next_item()` method.
- The driver can then process the item and send it to the DUT (usually through another method).
- The driver then acknowledges the item has been sent by calling `item_done()` from the `seq_item_port`.
- This procedure can continue (in a loop) until all items from the sequence have been sent.

There are other operation modes of the driver/sequencer but the "pull-down" procedure described above is by far the most common.

© Dolphin Technology Vietnam Center. All rights reserved



Every component must have a component utility macro (`uvm_component_utils`) and a component constructor with two arguments - `name` and `parent`.

Using a data type parameter on the `uvm_driver` class gives several advantages:

- A built-in handle of the data type, named `req`, is automatically created.
- The `get_next_item()` method call off the `seq_item_port` returns an instance of the data item directly.
- Better compile-time error checking.

Only the active path of a UVC can have type parameters, i.e., only driver and sequencer components, and sequence data items.

Getting and processing data items is carried out in a `run_phase()` task:

- The driver calls the `get_next_item()` task off `seq_item_port` to retrieve the next data item. The call execution blocks indefinitely until an item is provided.
- The driver then send the data item to the RTL design using a `send_to_dut()` task (details later in the course).
- Finally the driver calls the `item_done()` function to indicate to the sequencer that the driver is done processing the item. The call can optionally take a return data item as an argument.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

Sequencer

apb_master_sequencer.sv

```
class apb_master_sequencer extends uvm_sequencer #(apb_master_item);

  `uvm_component_utils(apb_master_sequencer)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

endclass : apb_master_sequencer
```

Extend uvm_sequencer

Parameterized driver needs a parameterized sequencer

Component utilities macro

Component constructor

By default, a squencer does not generate a stimulus

- A sequence data item defines transactions for a sequencer
- Many sequences can be defined for a given sequencer
- A sequence must be executed on the sequencer to apply its stimulus

Dolphin Confidential Information

Using the TLM sequencer/driver connection and sequences to define stimulus makes our sequencer declaration very small and simple.

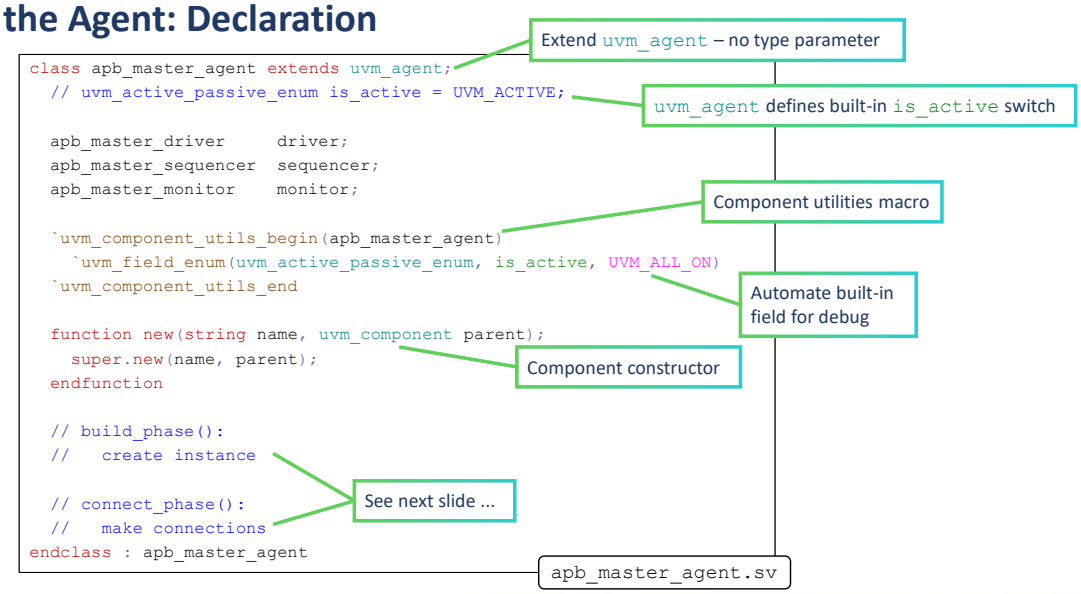
Note the communication methods (`get_next_item`, `item_done`) are defined in the `uvm_sequencer` subclass.

A sequencer does not create any transactions by default. A separate sequence data item defines the transaction traffic for a sequencer. There are typically many different sequences defined for a given sequencer, reflecting all the different transactions a UVC can create.

A simple configuration setting can be used to tell a sequencer which sequence to execute in `run_phase()` or a specific run sub-phase. Alternatively a sequence can be executed on a sequencer from within a test class.

© Dolphin Technology Vietnam Center. All rights reserved

Creating the Agent: Declaration



Dolphin Confidential Information

Agents and monitors cannot have type parameters.

By extended from `uvm_agent`, we inherit the `is_active` property to control the active/passive mode of the agent. `is_active` is of type `uvm_active_passive_enum` which is a built-in UVM enumerated type with values `UVM_ACTIVE` or `UVM_PASSIVE`. As `is_active` is not automated inside `uvm_agent`, we should add local automation within the begin/end form of our agent `uvm_component_utils` macro.

The `is_active` property is explicitly given a default value of `UVM_ACTIVE`. We will show you how to change the `is_active` value later in the Configurations module of the course.

The agent must contain the driver, sequencer and monitor components, for which we need handles. Instances for the handles will be created in the `build_phase()` method (not the constructor), and connected in the `connect_phase()` method. This is so hierarchical construction can be controlled by the UVM simulation phases.

Only the `is_active` property requires automation. As component instances, the `sequencer`, `driver` and `monitor` handles are automatically automated.

© Dolphin Technology Vietnam Center. All rights reserved

Creating the Agent: Build and Connect

```
class apb_master_agent extends uvm_agent;
  // uvm_active_passive_enum is_active = UVM_ACTIVE;

  // HANDLES, MACRO and CONSTRUCTOR declarations as previous slide

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    monitor = new("monitor", this);
    if (is_active == UVM_ACTIVE) begin
      sequencer = new("sequencer", this);
      driver = new("driver", this);
    end
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    if (is_active == UVM_ACTIVE)
      driver.seq_item_port.connect(sequencer.seq_item_export);
  endfunction
endclass : apb_master_agent
```

Inherited `is_active` switch

`super.build_phase` is essential

Agent sub-components created in build (sequencer & driver only if agent is active)

Connect sequencer and driver if agent is active

Connect method is called from the port and takes the export as an argument

`apb_master_agent.sv`

Dolphin Confidential Information

The sub-components of the agent must be created in a `build_phase()` function. The monitor component is always created, but the sequencer and driver care only created if the agent is active, i.e. the `is_active` property of the agent has the value of `UVM_ACTIVE`. The constructor calls for the components must have two arguments:

- A string matching the handle identifier for the name argument.
- The keyword `this` for the `parent` argument, indicating these components were created in this instance of the agent class.

In a `connect_phase()` function, we connect the driver `seq_item_port` to the sequencer `seq_item_export`, but only if the agent is active. The `connect` method is always called from the port object and takes the export as an argument.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

Creating and Instantiating the UVC Top Level (env)

The UVC top level uses the uvm_env (environment or env) subclass.

You should have the knowledge to create this …

```
class apb_env extends uvm_env;
  // agent handle
  // component utilities macro
  // component constructor:

  // build_phase():
  //   instantiate agent

endclass : apb_env
```

apb_env.sv

Entend uvm_env
no type parameter

```
class apb_uart_env extends uvm_env;
  ...
  // yapp_env handle
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // instantiate apb uvc
  endfunction
  ...
endclass : apb_uart_env
```

apb_uart_env.sv

Create APB UVC instance in environment build phase

Dolphin Confidential Information

The top level of the UVC is defined using the uvm_env class. env is short for environment, and this term is frequently used for describing the top level of the UVC. However, the env and environment terms are also used for other layers in a verification design (e.g., the testbench) so you need to be careful with its use.

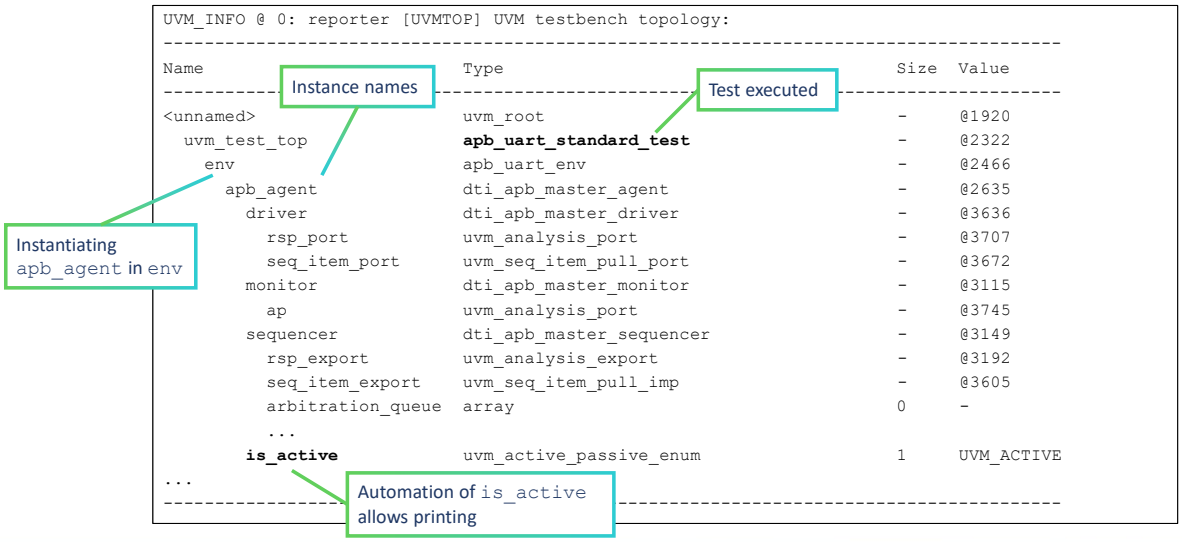
The UVC is a component, so don't forget to include a component utilities macro and component constructor.

The UVC must declare an agent handle, and construct an instance of the agent in the build phase.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

Topology with APB Agent Instantiation



Dolphin Confidential Information

A topology report provides a huge amount of useful information about your verification environment.

In Cadence's simulator, the simulation output, including the topology report, is saved to the file `xrun.log`.

The topology report can tell you:

- What test is running (type of `uvm_test_top`). Is this correct?
- The hierarchy of your test environment. Is every UVC present and is every UVC structure as expected?
- Starting value of key properties (e.g., `is_active`). Is every property printed? If not, you may have forgotten to automate the property to enable printing.

`uvm_top.print_topology()` is a useful debug construct for an UVM testbench. It allows the designer to check the hierarchy structure and property values of the testbench, and is key in understanding which test is running; which agents are active and which sequences are running. Always include this construct somewhere in your testbench.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

Reference: Type Parameterization

```
class apb_master_driver extends uvm_driver #(apb_master_item);
  ...
endclass : apb_master_driver

class apb_master_sequencer extends uvm_sequencer #(apb_master_item);
  ...
endclass : apb_master_sequencer

class apb_base_seq extends uvm_sequence #(apb_master_item);
  ...
endclass : apb_base_seq
```

```
class apb_master_monitor extends uvm_monitor;
  ...
endclass : apb_master_monitor

class apb_master_agent extends uvm_agent;
  ...
endclass : apb_master_agent

class apb_uart_env extends uvm_env;
  ...
endclass : apb_uart_env
```

Only the active path of a UVC has built-in type parameters

- Driver, sequencer, sequence

Other UVC classes do not

- Monitor, agent, uvc

Type parameters allow:

- Better compile-time checking
- Built-in `req` handle

Dolphin Confidential Information

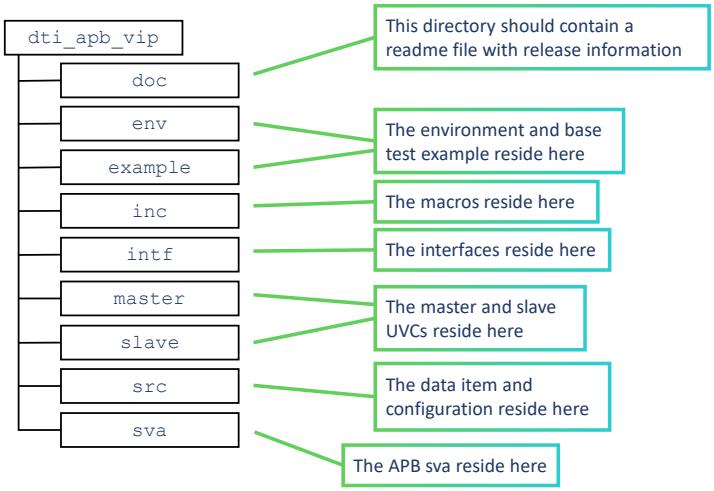
Only the active path of a UVC can have type parameters, i.e., only driver and sequencer components, and sequence data items. If you have a type parameter on any one of these classes, then each one must be type parameterized, i.e. a typed sequencer can only be connected to a typed driver and can only execute typed sequences.

© Dolphin Technology Vietnam Center. All rights reserved

Creating an Interface UVC

VIP Directory Structure

The VIP name is used for the "root" directory of the UVC files



© Dolphin Technology Vietnam Center. All rights reserved

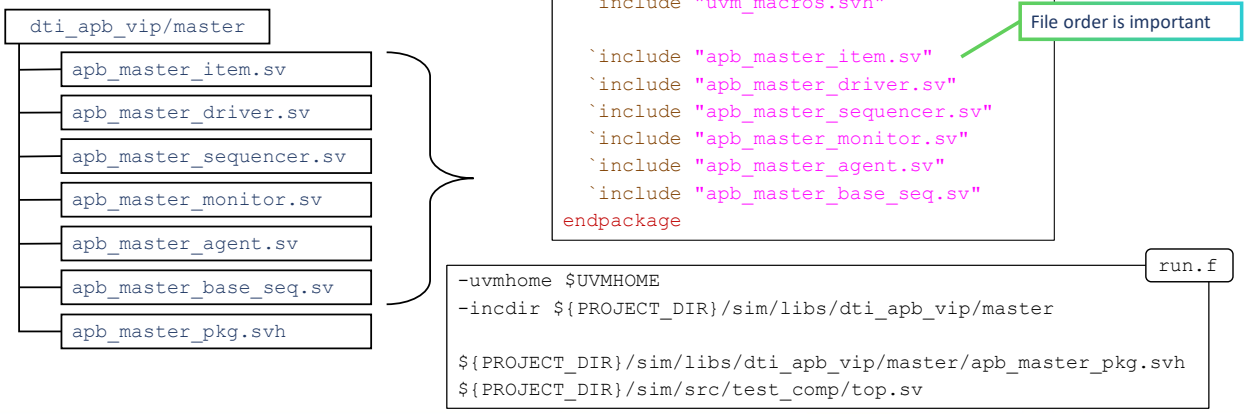
Creating an Interface UVC

UVC Package and Include Files

Each UVC has a single package that includes all the UVC base files.

- Users import this package to use UVC

Package resides in the master/slave subdirectory



Dolphin Confidential Information

All the sv files required by the UVC are included into a SystemVerilog package. As packages are compiled individually, the package must also contain an import of the UVM package and an include for the UVM macros header file.

The order of files includes is important and must be listed from bottom-up. This is due to declaration dependencies in the UVC hierarchy. You must include the data item first. You must include the monitor, driver and sequencer before the agent can be included. You must include the agent before the UVC env can be included.

Any other code not directly related to the UVC should be declared in the tb directory and be accessed separately, not as part of the UVC package.

© Dolphin Technology Vietnam Center. All rights reserved

08

Configuration



© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Use the UVM configuration mechanism to:
  - Control topology during the build, for example:
    - Set UVC agent active or passive
    - Set the number of UVC master and slave agents
  - Assign instance-specific properties for UVCs, for example:
    - Specifying illegal address ranges for stimulus

© Dolphin Technology Vietnam Center. All rights reserved

Configuring Topology

Topology can be configured according to property values

- For example, **`is_active`** property of the agent determines whether the sequencer and driver are created
- **`is_active`** is called a config property

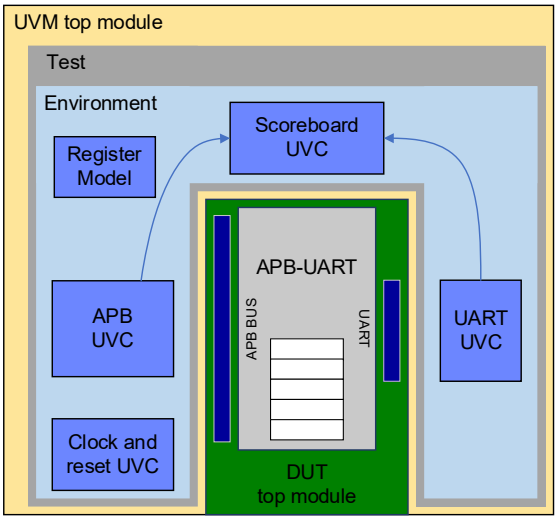
Need to control config properties from a higher level

- Typically, in a high-level testbench or test

Properties must be set before components are created

- For example, set agent passive before it is constructed

We do not want to pass property values from the top via arguments to `new()` or `build_phase()`



Dolphin Confidential Information

The testbench hierarchy can be configured by component properties. For example, in a UVC, every agent should have an is_active property which determines whether the agent is active or passive. When the agent is built the sequencer and driver components are created only if is_active has the value UVM_ACTIVE. Properties such as is_active are known as config, or configuration properties.

UVC configuration is usually controlled by higher-level components such as a environment or test. An environment will configure a UVC when it is instantiated. A test may reconfigure the hierarchy for a specific test.

In both cases, the configuration must be set up before the UVC components are created. If we set is_active to UVM_PASSIVE after the agent was built, it would be too late – the sequencer and driver would already be created.

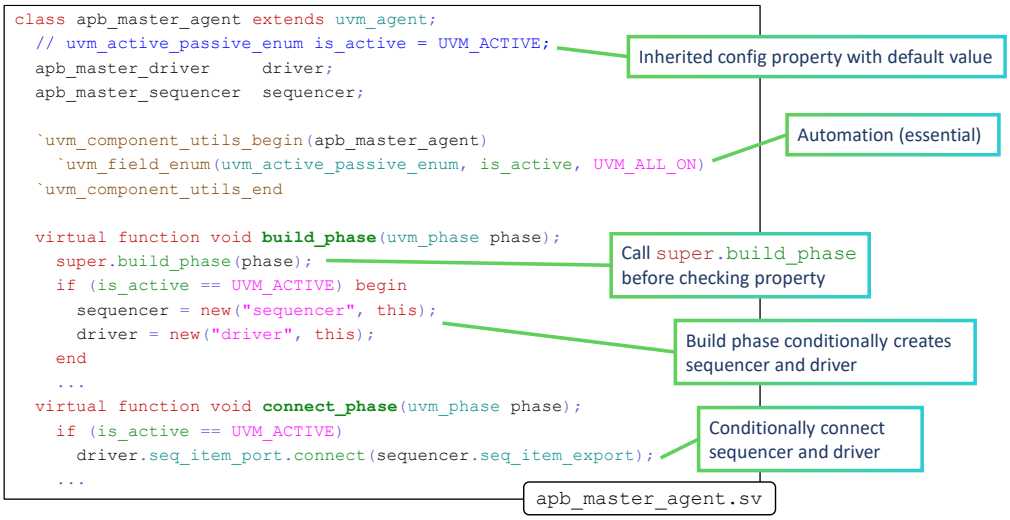
We could pass down config property values via arguments to the constructor or build phase methods. However if we have five UVCs, each with three config properties, then that is a large number of arguments to manage. Also if we swapped one UVC for another with different properties, the constructor or build phase method arguments would need to be rewritten throughout the hierarchy.

UVM uses a different mechanism to set config properties.

This is a standard problem in Object Oriented Design, and configuration used with build phases is a standard solution (called a design pattern).

© Dolphin Technology Vietnam Center. All rights reserved

Review – is_active Agent Config Property



Dolphin Confidential Information

This is the same agent code as we showed previously in the module on Creating a Simple UVC, except the monitor code is not shown.

The config property `is_active` is used to conditionally build the sequencer and driver instances, and to conditionally connect driver and sequencer. The default value of the property is set to `UVM_ACTIVE` in the declaration, so how can we change the value of `is_active` to set the agent to passive? We shall see how to do this in the following slides.

There are two important points to note when using config properties:

1. The property is automated, i.e., it has a field automation macro.

2. In the build phase method, `super.build_phase()` is called first before we test the value of `is_active`.

The following slides will explain the significance of these points in more detail.

© Dolphin Technology Vietnam Center. All rights reserved

Setting Config Properties

UVM provide a configuration mechanism to set property values from higher levels:

```
uvm_config_db#(<type>)::set(<context>, <instance>, <field>, <value>);
```

Where:

- `value` is the desired property value
- `field` is the config property name (string)
- `instance` is the hierarchy of instance names to the property component (string)
- `context` is the scope for resolving `instance` (usually `this`)
- `set` method is static
- `type` is the config property type

For simple config settings, all you have to do is call `set`

- The config is automatically resolved in the build phase

Dolphin Confidential Information

Config property values are assigned using the `set` method of the configuration database.

Syntax for configuration database `set` method:

```
static function void set(uvm_component cntxt, string inst_name,
string field_name, T value)
```

where

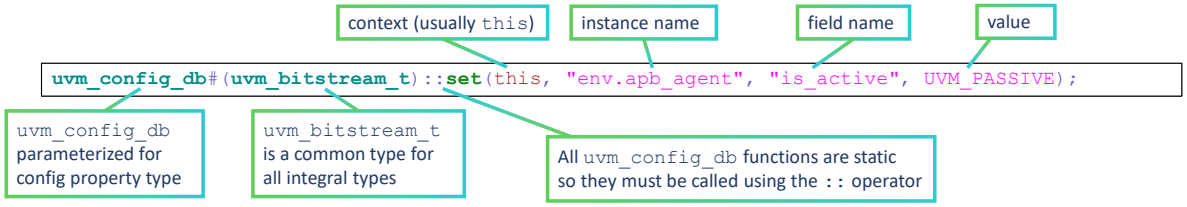
- `cntxt` is a context (usually assigned to this) from where the relative pathname of `inst_name` is resolved.
- `inst_name` is a string concatenation of component instance names representing a relative pathname to a specific component instance.
- `field_name` is the string name of a config property in the specific component instance.
- `value` is a literal of a compatible type for the config property.

`set` is a static method called from `uvm_config_db` which must be type parameterized with a type name compatible with the type of the property `field_name` and value.

If constructed correctly, config settings are automatically resolved in the build phase.

© Dolphin Technology Vietnam Center. All rights reserved

Config Property Setting Example



- Database must have a type parameter
  - Each unique type parameter creates a separate database specialization
  - Allows compile time checking and debug
  - Allows configuration for user-defined types such as virtual interfaces
  - UVM has convenience types for common `uvm_config_db` parameterizations

```
uvm_config_int = uvm_config_db#(uvm_bitstream_t)
```

- Property pathname is a concatenation of context, instance, and field

Dolphin Confidential Information

Here is an example of setting a UVC agent to passive by assigning the value `UVM_PASSIVE` to the `is_active` property of the component at the instance pathname `env.apb_agent`.

The configuration database must be parameterized for the type of the config property. This effectively creates a separate configuration database for that type. Type parameters allow config databases to be created for any standard or user-defined type and allows better compile time checking. However the type used in the database `set` must match the type of the property, the type of the value and the type of the operation to assign the config property.

Type matching can be a particular issue for integral types (e.g. `int`, `integer`, `byte`, `logic`, `bit`) as separate config databases are created for each named type. Therefore UVM defines a common integral type, `uvm_bitstream_t`, for all integral configuration accesses.

```
typedef logic signed [4095:0] uvm_bitstream_t;
```

There are several typedefs declared for common `uvm_config_db` parameterizations:

```
typedef uvm_config_db#(uvm_bitstream_t) uvm_config_int;
typedef uvm_config_db#(string) uvm_config_string;
typedef uvm_config_db#(uvm_object) uvm_config_object;
```

The property pathname is constructed by concatenating the `context`, `instance` and `field` arguments into an absolute hierarchical pathname.

© Dolphin Technology Vietnam Center. All rights reserved

How Configuration Works

```
uvm_config_int::set(this, "env.apb_agent", "is_active", UVM_PASSIVE);
```

Context	Instance	Field	Value
this	"env.apb_agent"	"is_active"	UVM_PASSIVE

Set method registers the desired setting in the configuration database

- Method must be called before the property component is built
  - This is why instance and field names are strings

Setting applied by `apply_config_settings` method in the build phase

- Called automatically by `super.build_phase()`
  - Must be called *before* checking the config property value
- The config database is searched for a matching pathname and field name
- Settings are applied *only* when match is found
  - Errors in instance and field names will leave properties un-set!

Dolphin Confidential Information

The `set` method writes the desired configuration setting into a configuration database. You must call the method before the component containing the affected configuration property is built, so that the setting can be propagated when the component is created. If the `set` method is called after the component is created, the setting will not take effect. The requirement to make the setting before the component is built explains why both the instance and field name arguments of set config methods are strings. Neither the instance pathname nor the property field name exist when the methods must be called.

Settings are applied by the method `apply_config_settings()`. This is called automatically by a call to `super.build_phase()` in the build phase method of any component. `apply_config_settings` checks the path name of the current component against every entry in the configuration database. If a pathname match is found, it looks for a field name match, and if one is found, the property is assigned the value from the database.

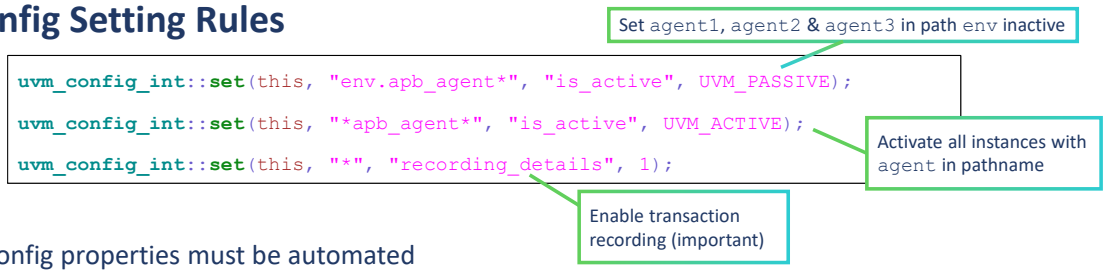
The user can also (and sometimes has to) explicitly get configurations using a method call.

Obviously the `super.build_phase()` call must be made before a config property value is used. If no match is found in the instance name or field name, then the configuration setting is not applied. There is no warning or error if a setting does not match, therefore errors in either the instance or field name will leave config properties unset and these can be difficult to debug.

© Dolphin Technology Vietnam Center. All rights reserved

Configuration

Config Setting Rules



- Config properties must be automated
- Instance and field arguments must match hierarchy
- Instance *only* can include * or ? wildcard characters, or regular expressions
  - Allow one setting for multiple properties in same or different components
- Config settings in higher scope take precedence over lower scopes
  - For example, test class set overrides a testbench set
- Config settings in the same scope conform to "last one in wins"

© Dolphin Technology Vietnam Center. All rights reserved

Debugging Configuration Settings

- Configuration can fail
  - Typically mismatches in instance or field arguments
- By default, unmatched configurations are not reported
- Unmatched settings can be printed at end of simulation
  - Prints arguments of config settings which are unmatched
  - Compare pathname in report (used in `set`) with pathname from topology report

```
class apb_uart_base_test extends uvm_test;
  ...
  function void check_phase(uvm_phase phase);
    check_config_usage();
  endfunction
...
endclass
```

```
UVM_INFO @ 1502900000: uvm_test_top [CFGNRD]  ::: The following resources have at least one write and no reads :::
UVM_INFO /tools/uvm/1800.2-2020.3.0/src/base/uvm_resource_pool.svh(1292) @ 1502900000: reporter
[UVM/RESOURCE_POOL/PRINT_QUEUE] (uvm_queue#(uvm_resource_base)@6508) {
  [0]: {
    name: is_active
    value: 1
    scope: uvm_test_top.env.agent
    precedence: 'd999
    accesses: {
      [uvm_test_top]: reads: 0 @ 0  writes: 1 @ 0
    }
  }
}
```

Field name

Value

Component pathname (regexp syntax)

Type

Config written (set) but not read (matched)

Dolphin Confidential Information

The most common config setting problems are:

- Forgetting to automate properties to which config settings are applied.
- Mismatches in instance or field strings due to typos or missing path references.
  - Remember the wildcard symbol `*` represents *one* or more characters, and `?` represents exactly 1 character.
- Type mismatches between method calls and properties.

In many cases config settings will not be applied and no errors or warnings are generated.

Configurations can be checked by calling `check_config_usage();`

```
function void check_config_usage (bit recurse = 1);
```

This reports settings in the configuration table which have been written (set) but not read (matched).

However, `check_config_usage()` does NOT generate mismatch errors for un-automated config properties, even though these still fail.

© Dolphin Technology Vietnam Center. All rights reserved

Selected Configuration Database Methods

Method	Description	Syntax
set	Writes an entry setting `field_name` in the path `cntxt.inst_name` to value.	`static function void set (uvm_component cntxt, string inst_name, string field_name, T value);`
get	Reads the entry for `field_name` in the path `cntxt.inst_name`. Assigns `value` and returns 1 if the entry exists, 0 otherwise.	`static function bit get (uvm_component cntxt, string inst_name, string field_name, inout T value);`
exists	Returns 1 if a config setting exists for `field_name` in the path `cntxt.inst_name`, 0 otherwise. Set `spell_chk` to 1 to enable spell checking if the field should exist.	`static function bit exists (uvm_component cntxt, string inst_name, string field_name, bit spell_chk = 0);`
wait_modified	Blocks until a new configuration setting is applied for `field_name` in the path `cntxt.inst_name`.	`static task wait_modified (uvm_component cntxt, string inst_name, string field_name);`

An explicit manual `get` may be required when:

- `set` is called from the top-level module or outside the build phase
- Property type does not have an automation macro

See Connecting to a DUT module

Dolphin Confidential Information

Argument descriptions are as follows:

- `cntxt` – component context from which the set is being done. Usually mapped to `this`. `get_full_name()` of the context is implicitly added to the instance name.
- `inst_name` – pathname to the target component instance from the context. This name may contain wildcards or regular expression syntax .
- `field_name` – target field in the target instance. From UVM1.2 the field cannot contain wildcards.
- `value` – configuration value for the field name at this instance.

© Dolphin Technology Vietnam Center. All rights reserved

set_config Methods

There is a simpler alternative for setting integral or string config properties.

- Based on `set_config_*` method

```
virtual function void set_config_* (string inst_name,
                                    string field,
                                    bitstream_t value);
```

  - Where `*` is `int`, `string` or `object`, depending on type of config property
- Has same arguments as uvm_config_db `set` except for context
  - Context is set to the scope where `set_config` is called
- However, `set_config` calls are deprecated in UVM1.2
  - Will be removed from a future release
  - Should be avoided in new UVM code
  - Consider converting any use in legacy environments to database calls

Dolphin Confidential Information

`uvm_config_db` has better debugging and compile-time checking than the simple `set_config` method call. However it is more complex to use, and the compile time checking comes at the expense of strong type sensitivity, which can cause problems in use.

However in UVM1.2, the set_config method calls are marked as deprecated, which means they will be removed from a future version of UVM. Therefore although you may see these calls in legacy code, you should avoid using them in new designs.

© Dolphin Technology Vietnam Center. All rights reserved

09

Type Overrides and the Factory



© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Change the default behavior of any data item or component using the UVM factory and override methods

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Adding Different Behavior to Existing Objects

Need to modify behavior for many scenarios:

- Supporting test requirements
  - For example, changing default data randomization:
    - Replace, remove or layer constraints
- Reuse of Verification IP
  - For example, changing procedural behavior:
    - Customizing driver protocol for specific tests
    - Modifying monitor checking algorithm for error recovery
- Module-to-system reuse
  - For example, adding more properties to a class
    - Extracting more information from a scoreboard

Dolphin Confidential Information

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Sequence Item Layering

- `sysclk_seq_item` is derived from `dti_sysclk_seq_item` in the VIP.
- Object-oriented programming allows for deriving a new subclass from the parent class and adding new properties
- But how do we use these sub-classes?

```
class dti_sysclk_seq_item extends uvm_sequence_item;
  `uvm_object_utils(dti_sysclk_seq_item)

  function new(string name);
    super.new(name);
  endfunction

  virtual function string toString();
    return "Base dti_sysclk_seq_item";
  endfunction
endclass
```

```
class sysclk_seq_item extends dti_sysclk_seq_item;
  `uvm_object_utils(sysclk_seq_item)

  rand reset_n;
  rand SIG_BEH apb_clk;
  rand time apb_period;
  rand SIG_BEH uart_clk;
  rand time uart_period;

  function new(string name);
    super.new(name);
  endfunction

  virtual function string toString();
    string str = "";
    str = $psprintf("%sduration    = %1d\n", str, duration);
    str = $psprintf("%sreset_n     = %s\n", str, reset_n);
    str = $psprintf("%sapb_clk     = %s\n", str, apb_clk);
    str = $psprintf("%suart_clk    = %s\n", str, uart_clk);
    return str;
  endfunction
endclass
```

Creates a subclass that extends from the base class

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Sequence Item Layering Problem

Both the `dti_sysclk_seq_item` base class and `sysclk_seq_item` subclass must contain an object utilities macro. Both must also contain an object constructor.

```
class dti_sysclk_drv extends uvm_driver #(dti_sysclk_seq_item);
  `uvm_object_utils(dti_sysclk_drv)
  // dti_sysclk_seq_item req;
  ...
endclass
```

In multiple locations...

```
...
dti_sysclk_seq_item req = new("req");
...
```

...will need to change `dti_sysclk_seq_item` to `sysclk_seq_item`

```
class sysclk_driver extends dti_sysclk_drv;
  `uvm_object_utils(sysclk_driver)
  // use properties of sysclk_seq_item
  ...
endclass
```

`sysclk_driver` needs to use `sysclk_seq_item`

```
class sysclk_seq_item extends dti_sysclk_seq_item;
  `uvm_object_utils(sysclk_seq_item)
  rand reset_n;
  rand SIG_BEH apb_clk;
  rand time apb_period;
  rand SIG_BEH uart_clk;
  rand time uart_period;
  ...
endclass
```

```
class dti_sysclk_agent extends uvm_agent;
  `uvm_object_utils(dti_sysclk_agent)
  dti_sysclk_drv drv;
  ...
endclass
```

The agent needs use new driver

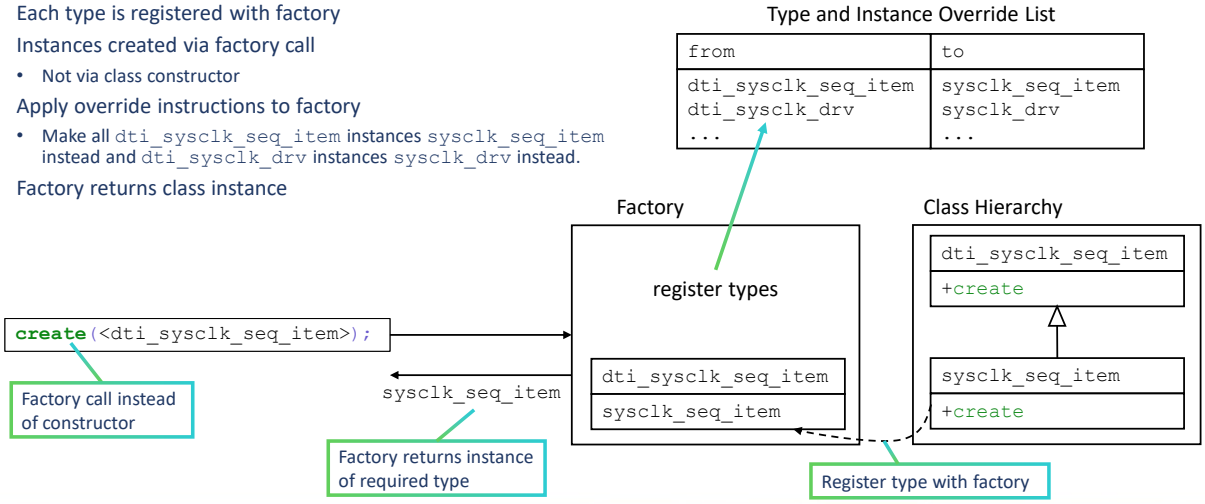
© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Solution: Factory

Central location to create class instances

- Each type is registered with factory
- Instances created via factory call
  - Not via class constructor
- Apply override instructions to factory
  - Make all `dti_sysclk_seq_item` instances `sysclk_seq_item` instead and `dti_sysclk_drv` instances `sysclk_drv` instead.
- Factory returns class instance



Dolphin Confidential Information

Creating objects of the correct type in a reusable, flexible fashion is a standard problem in objectoriented design, with a standard solution called a factory. A factory is a class which creates object instances of the required type from a family or hierarchy of related classes.

Calls to a class constructor are replaced with calls to a factory creator method. The factory creates the required class instance and returns a handle to the instance. The handle is of the base type for that class hierarchy, allowing the factory to return any subclass of the base type.

As further subclasses of a type are developed, they can be easily added to the factory for creation. In some implementations, the creation methods are included in the subclass declarations, and the factory is a standard “interface” between the constructor code and the subclass creation methods.

Factories allow greater flexibility for obtaining an object instance of a particular type. For example, instead of simply calling a constructor to create a new object, the factory could select the object from an existing pool of resources; apply complex configuration on the object, etc.

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

UVM Factory

Operation of the factory requires:

1. Registration of all UVM types with the factory
   - Utility macros register types with the factory
     - `` `uvm_object_utils(class_name) ``
     - `` `uvm_component_utils(class_name) ``
2. Using a factory create call for instances
   - Utility macros define `create()` as static method of class
   - Make sure all data and component class instances use `create()` rather than `new()`
     - Some exceptions (e.g., TLM connection objects – see later)
3. Global and instance-specific type override instructions for factory

Dolphin Confidential Information

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Object Factory Use: create

```
class dti_sysclk_seq_item extends uvm_sequence_item;
  `uvm_object_utils(dti_sysclk_seq_item)

  function new(string name);
    super.new(name);
  endfunction
  ...
```

add dti_sysclk_seq_item to factory

```
class dti_sysclk_monitor extends uvm_monitor;
  `uvm_object_utils(dti_sysclk_monitor)
  dti_sysclk_seq_item item;
  ...
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  task run_phase(uvm_phase phase);
    item = new("item");
    item = dti_sysclk_seq_item::type_id::create("item");
...
```

Instead of new...

... use create instead

Instance name

```
class dti_sysclk_agent extends uvm_agent;
  `uvm_object_utils(dti_sysclk_agent)
  dti_sysclk_monitor monitor;
  ...
  function void build_phase(uvm_phase phase);
    monitor = new("monitor", this);
    monitor = dti_sysclk_monitor::type_id::create("monitor", this);
  endtask
endclass
```

Handle type

Static create call

Instance name and parent pointer

Almost *all* instances should be made by calling factory `create` instead of constructor `new`

Dolphin Confidential Information

If the class does not have a utility macro, then the create call will flag a compilation error.

If we do not use the factory create call for a class instance, then factory type overrides cannot be used which means the class cannot be modified through the use of a subclass declaration. In most cases we want to allow this, so almost all instances should be made with create calls. There are exceptions to this rule we will see later (for example, TLM connector objects which are introduced in the Building a Scoreboard module).

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Type Overrides with Factory

```
class apb_uart_base_test extends uvm_test;
  `uvm_object_utils(apb_uart_base_test)
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    set_type_override_by_type(dti_sysclk_seq_item::get_type(), sysclk_seq_item::get_type());
  endtask
...
```

Factory override creates `sysclk_seq_item` instead of `dti_sysclk_seq_item`

Static method calls allow compiler checking

```
...
  item = dti_sysclk_seq_item::type_id::create("item");
...
```

Every create for `dti_sysclk_seq_item` now returns a `sysclk_seq_item`

Dolphin Confidential Information

`set_type_override_by_type` uses types for the original and override type identifiers. The type identifiers can be obtained using `get_type()` method calls, as in the example above.

© Dolphin Technology Vietnam Center. All rights reserved

Type Overrides and the Factory

Notes About the Factory

Overrides can be called at any time

- Overrides apply for all subsequent calls to the factory

You don't have to instantiate the factory

- It is auto instantiated on first registration of an object

For debug, you can print the factory contents:

- Prints registered types and overrides

UVM1.1

```
factory.print();
```

UVM1.2

```
uvm_factory my_f = uvm_factory::get();
my_f.print();
```

Dolphin Confidential Information

`function void print (int all_types=1)`

Prints the state of the `uvm_factory`, depending on the value of the argument `all_types`:

- When 0, only type and instance overrides are displayed.
- When 1 (default), all registered user-defined types are printed as well.
- When 2, the UVM types (prefixed with `uvm_`) are included in the list of registered types.

© Dolphin Technology Vietnam Center. All rights reserved

10

Sequences



© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Use UVM macros and methods to create sequences to implement common stimulus requirements
  - Simple randomized data
  - Directed tests
  - Stimulus controlled by properties
  - Hierarchical stimulus
- Define which sequence is executed by the sequencer
- Use the UVM objection mechanism to control the end of simulation

© Dolphin Technology Vietnam Center. All rights reserved

What are Sequences?

A sequence is a set of transactions that accomplish a defined task for the DUT

- Typically, tests are described at the transaction level
  - Easier to write, maintain, and control
  - Examples: instructions, packets
- Single items cannot capture the high-level intention
  - Ordered stream of transactions is required
  - Examples: program generation, connection-based protocols

Some sequences should be provided as part of a reusable component...

- Capture important scenarios that should be exercised
- Provide examples for a test writer to extend from or reuse

...however, a test writer will frequently have to create additional sequences

Dolphin Confidential Information

© Dolphin Technology Vietnam Center. All rights reserved

UVM Sequence Requirements

- Easy to write and use
  - No need to learn complex mechanism
  - Minimize the protocol/environment knowledge requirements
- Combine declarative and procedural constraints
- Can be generated on-the-fly or at time zero
- Can nest sequences
- Can be timed as well as procedural
  - For acceleration, delays should be mapped to interface methods (see later)
- Ability to create a system-level sequence – drive multiple interfaces from a single sequence (described later)

Dolphin Confidential Information

UVM sequences need to be easy to use without complex syntax or methodology overheads. They need to be easy to use so that test writer, with only a minimal knowledge of the test environment and the stimulus protocol, can use and create sequences.

Sequences need to be able to combine the default declarative constraints which apply to every data item (e.g., declared in the `apb_item` class) with inline procedural constraints to modify individual data items.

Sequences should support the generation of data items as required during the run phase, perhaps in response to data from the DUT, or the generation and storage of thousands of data items at the beginning of the simulation, which is a technique for hardware acceleration and emulation.

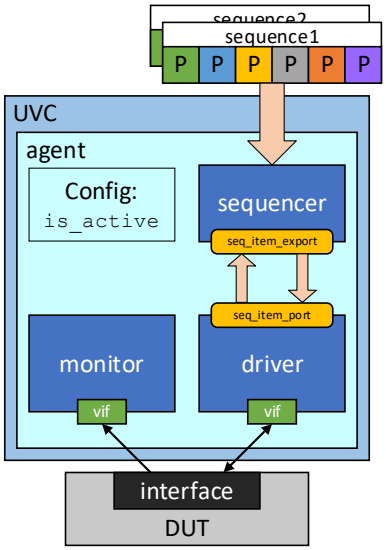
Sequences must support nesting to allow complex sequences to be built out of simple sequence building blocks.

Sequences must support timed behavior, such as delays, event expressions and parallelism (through fork-join blocks) as well as procedural behavior such as looping, conditional branching and iteration. However for acceleration efficiency, explicit delays and event expressions should be executed in the hardware partition only. A sequence should trigger a driver to execute interface methods to, for example, count clocks or wait for DUT events.

Finally we need to create sequences which control more than more UVC, to synchronize stimulus and behavior for all the inputs to the DUT. System-level (or virtual) sequences will be covered later in the class.

© Dolphin Technology Vietnam Center. All rights reserved

Sequences and UVCs Review



Sequence – data item

- Defines stimulus pattern of individual items (APB packets)
- Can contain declarations, procedural code, timing, constraint blocks, etc.
- Can reuse other predefined sequences
- Provides a standard test writer interface

Sequencer – component

- Provides randomization of sequences and items
- Provides TLM connection to a driver
- Executes sequence by supplying individual items (APB packets) to driver on demand

Driver – component

- Provides TLM connection to a sequencer
- Request the next item (APB packet) from sequencer
- Implements the protocol for driving the item on the interface

© Dolphin Technology Vietnam Center. All rights reserved

UVM Sequences Example

```
//2 random frames, second frame is 'h17
class uart_tx_seq extends uvm_sequence #(uart_tx_item);
  // uart_tx_item req;

  `uvm_object_utils(uart_tx_seq)
  `uvm_declare_p_sequencer(uart_tx_sequencer)

  function new (string name = "uart_tx_seq");
    super.new(name);
  endfunction

  virtual task body();
    `uvm_info(get_type_name(), "uart_tx_seq", UVM_LOW)
    `uvm_do(req)
    `uvm_do_with( req, {uart_tx_data_frame == 'h17;} )
  endtask
endclass
```

Extend from `uvm_sequence` with type parameter

Parameterization defines built-in `uart_tx_item` handle `req`

Register sequence with factory

Standard data constructor

Specify `body()` task with `` `uvm_do `` macros

Useful to identify sequence

2 random frames, second with constraint on packet `uart_tx_data_frame` field

Sequences are data items with a data constructor and object utility macro

Dolphin Confidential Information

The sequence has a type parameter to match the type parameter on its associated sequencer and connected driver. Type parameterization defines a built-in handle on the type named `req`.

The type parameter (`uart_tx_item`) must be of a type which derives from `uvm_sequence_item` in order to use `uvm_do` macros.

Remember a sequence is a data item. Therefore it has a data constructor with single `name` argument. A sequence uses the standard object utilities macro to register the sequence with the factory.

The stimulus provided by the sequence is defined in a `body()` task. Inside the task we typically use UVM `do` macro variants to define the stimulus. This sequence creates one random packet with the default `uart_tx_item` constraints (equivalent to `req.randomize()`), followed by a second randomized packet with a constraint specifying the `uart_tx_data_frame` field of the packet equals zero (equivalent to `req.randomize() with {uart_tx_data_frame == 'h17;}`)

However the do macros carry out much more functionality than just randomizing the packet, as we shall see on the following slides.

© Dolphin Technology Vietnam Center. All rights reserved

The do Operation

```
...
  task body();
    `uvm_do_with( req, {uart_tx_data_frame == 'h17;} )
  endtask
```

Translates into:

- Allocate using the factory — Factory support out-of-the-box
- Wait till the item is needed — Reactive generation
- Randomize (with constraints or not) — Combines procedural (inline) and declarative (layered) constraints for extra flexibility
- Put on the consumer interface — Compatible with built-in TLM connection
- Block code execution till `item_done()` — Allows modeling timing, for example, scenario that calls for a frame, waits for FIFO full, and sends an illegal frame

Dolphin Confidential Information

The `uvm_do` macros allow you to easily combine declarative (layered) constraints from the data item descriptions with procedural (inline) constraints in the macro argument list. In this way, the declarative constraints which affect every data item instance can be combined with procedural constraints which affect individual instances, to create the stimulus you need.

© Dolphin Technology Vietnam Center. All rights reserved

`uvm_do Macros: What Do They Do?

Step	Summary	Description	Methods
1	creation	Create instance, set parent and link to sequencer	`create`
2	synchronize	Wait for sequencer `get_next_item()` request	`start_item`
3	pre_do hook	Executes the `pre_do()` task of the sequence	
4	randomization	Randomize the instance. If randomization fails, a warning is issued	`randomize`
5	mid_do hook	Executes the `mid_do()` function of the sequence	`finish_item`
6	send item and wait until done	For items: send item to sequencer, and wait until item done For subsequences: execute the sequence `body()` method	
7	post_do hook	Execute the `post_do()` function of the sequence	

Dolphin Confidential Information

The `uvm_do` macro performs seven steps to create and randomize an item, and synchronize with the sequencer to pass the item to the driver under the control of the get_next_item/item_done method calls.

There are various ways to customize the behavior of the `uvm_do` macro. One (less common) option is to use hook methods (similar to pre/post-randomise). If you define the hook method for the sequence, it is automatically executed at a defined point in the macro execution.

A more common customization option is to replace `uvm_do` with other macro variants (see next slide) or to use method calls instead (see later). These perform the same actions as `uvm_do`, but allow us to insert custom code between various steps in the flow.

A warning (`` `uvm_warning``) is issued if randomization fails due to constraint errors, *but* the simulation continues with a non-randomized data item.

© Dolphin Technology Vietnam Center. All rights reserved

Alternative to uvm_do Macros: Explicit Flow with Methods

```
// single frame is 'h17
class uart_tx_seq extends uvm_sequence #(uart_tx_item);

  `uvm_object_utils(uart_tx_seq)
  `uvm_declare_p_sequencer(uart_tx_sequencer)

  function new (string name = "uart_tx_seq");
    super.new(name);
  endfunction

  virtual task body();
    req = uart_tx_item::type_id::create("req");
    start_item(req);
    aserrt(req.randomize() with {uart_tx_data_frame == 'h17;});
    finish_item(req);
  endtask
endclass
```

Explicit creattion

create call in data item, not component, so no this

start_item is equivalent to steps 2-3 of `uvm_do

Explicit randomization

finish_item is equivalent to steps 5-7 of `uvm_do

Sequences are data items with a data constructor and object utility macro

Dolphin Confidential Information

Some people prefer to use the explicit flow instead of variations of the macros. UVM provides several ways to accomplish the same result and give full control of data creation using the sequence interface.

The explicit flow requires explicit creation of the data item instance which should be via a factory `create` call. However, the `create` call cannot use `this` as the second parent argument.

Parent arguments are only applicable in component classes, where they are used to locate the instance in the component hierarchy. As we are calling create in a data item (sequence `uart_tx_seq`) there is no component hierarchy, so the argument is not passed (it defaults to `null`).

If you do include the `this` argument in the `create` call of a data item you get a type compatibility compile error.

© Dolphin Technology Vietnam Center. All rights reserved

Additional `uvm_do Macros

For full flexibility, additional macros can be used for item and sequence execution.

Syntax

- `` `uvm_*(variable) ``
- `` `uvm_*_with(variable, { constraint-block } ) ``

`` `uvm_do ``	Performs all seven steps from previous slide (creation to `post_do`)
`` `uvm_do_with ``	Same as `` `uvm_do ``, but adds in-line constraints during randomization (`randomize with {}`)

`` `uvm_create ``	Only performs step 1 (creation)
`` `uvm_send ``	Performs steps 2-7 (synchronize to `post_do`), but skips step 4 (randomization)
`` `uvm_rand_send ``	Performs steps 2-7 (synchronise to `post_do`)
`` `uvm_rand_send_with ``	Performs steps 2-7 (synchronize to `post_do`) and adds additional constraints for randomization (step 4)

Dolphin Confidential Information

Syntax

- `` `uvm_do(variable) ``
- `` `uvm_do_with(variable, { constraint-block} ) ``
- `` `uvm_create(variable) ``
- `` `uvm_send(variable) ``
- `` `uvm_rand_send(variable) ``
- `` `uvm_rand_send_with(variable, { constraint-block} ) ``

The variable must be derived from `uvm_sequence_item` or `uvm_sequence`.

© Dolphin Technology Vietnam Center. All rights reserved

Directed Stimulus with Macros

```
// single frame is 'h17
class uart_tx_seq extends uvm_sequence #(uart_tx_item);

  `uvm_object_utils(uart_tx_seq)
  `uvm_declare_p_sequencer(uart_tx_sequencer)
  // constructor
  ...

  task body();
    req = uart_tx_item::type_id::create("req");
    `uvm_create(req)
    aserrt(req.randomize());
    req.uart_tx_data_frame == 'h17;
    `uvm_send(req)
  endtask
endclass
```

Create instance

Randomize instance to create legal data

Overwrite and updates fields as required

Send without further randomization

Create and send macros are ideal for defining directed or semi-directed stimulus

Dolphin Confidential Information

Create and send macro combinations can be useful for directed or semi-directed stimulus.

For example:

- Create a data instance using `` `uvm_create() ``.
- Optionally randomize the instance to create legal data.
- Overwrite the required fields of the instance (here to set the data frame).
- Send the instance without further randomization using `` `uvm_send() ``.

© Dolphin Technology Vietnam Center. All rights reserved

Sequence Properties

```
// 2 frames with data lower than 'h55
class uart_tx_seq extends uvm_sequence #(uart_tx_item);

  `uvm_object_utils(uart_tx_seq)
  `uvm_declare_p_sequencer(uart_tx_sequencer)

  rand bit [7:0] sdata;
  constraint c1 { sdata < 'h55; }

  // constructor
  ...

  task body();
    `uvm_do_with(req, {uart_tx_data_frame == sdata;})
    `uvm_do_with(req, {uart_tx_data_frame == sdata + 'h55;})
  endtask
endclass
```

`rand` sequence property randomized each time sequence called

Constraint for property randomization

Property used as constraints for packet generation

Dolphin Confidential Information

Note that a sequence can have properties. Properties can be used in the definition of the sequence behaviour. If the sequence property is declared as rand, then the property is randomized when the sequence is executed. The property can then be used anywhere in the `body()` task to generate stimulus, for example:

- In a repeat loop to create a variable number of packets.
- In `do_with` constraint blocks to define randomization constraints.

Constraints can be declared either in the sequence or in-line with the sequence call, to control the randomization of the sequence property.

© Dolphin Technology Vietnam Center. All rights reserved

Nesting Sequences

```
// call uart_tx_seq sequence wrapped with random packets
class uart_nested_seq extends uvm_sequence #(uart_tx_item);

  `uvm_object_utils(uart_nested_seq)
  `uvm_declare_p_sequencer(uart_tx_sequencer)

  uart_tx_seq tx_seq;
  // constructor
  ...

  virtual task body();
    `uvm_do_with(req)
    `uvm_do_with(tx_seq, {sdata == 'h55;})
    `uvm_do_with(req)
  endtask
endclass
```

Handle for previously defined sequence
Choose meaningful name for debug!

Constraint on `uart_tx_seq` sequence `rand` property

Dolphin Confidential Information

Sequences can be nested, for example to build up more complex stimulus from basic sequences.

A handle must be defined for the nested sequence, must be previously declared. It's a good idea to choose a meaningful name for the handle, related to the nested sequence, as this can help in debug (where only the handle name may be shown, not the nested sequence type).

A `uvm_do` macro can then be called on the sequence handle to execute it as a subsequence. The `uvm_do_with` macro variant can be used to pass inline constraints to the properties of the subsequence.

© Dolphin Technology Vietnam Center. All rights reserved

Sequence Execution During Simulation Run Time

```
class uart_nested_seq extends ...
  ...
  uart_tx_seq tx_seq;

  task body();
    `uvm_do_with(req)
    `uvm_do_with(tx_seq)
    `uvm_do_with(req)
  endtask
endclass
```

1

4

```
class uart_tx_seq extends ...
  ...
  rand bit [7:0] sdata;
  constraint c1 { sdata < 'h55; }

  task body();
    `uvm_do_with(req, {uart_tx_data_frame == sdata;})
    `uvm_do_with(req, {uart_tx_data_frame == sdata + 'h55;})
  endtask
endclass
```

2

3

© Dolphin Technology Vietnam Center. All rights reserved

Excuting Sequences

How do we control which sequences run on a UVC sequencer and when?

Several choices:

1. `run_phase()` (and every run sub-phase) has a `default_sequence` property
   - Set `default_sequence` to a sequence to execute it in that phase
2. Execute sequences directly on a UVC sequencer from a test class
3. Sequences can be grouped together into a sequence library
   - Not covered in this course

© Dolphin Technology Vietnam Center. All rights reserved

Run Phase Default Sequence

```
class mytest extends base_test;
  // macro and constructor
  ...
  function void build_phase(uvm_phase phase);
    uvm_config_wrapper::set(this,
                            "env.uart_tx_agent.sequencer.run_phase",
                            "default_sequence",
                            uart_tx_seq::get_type());
    super.build_phase(phase);
  endfunction : build_phase
  ...
endclass
```

This sequencer...

... in the run phase ...

... has a default sequence setting ...

... of this sequence

Dolphin Confidential Information

Every UVC has a `default_sequence` property for the `run_phase` and for every run sub-phase. By default the pointer is null.

A standard `uvm_config_db` call can be used to set the property to a specific sequence. That sequence is then executed by the UVC sequencer in the defined phase.

The type parameter for the `uvm_config_db` call is `uvm_object_wrapper`. This is a factory type declaration representing an abstract interface for all `uvm_object-based` and `uvm_component_based` objects which are registered with `uvm_factory`.

For convenience, a predefined `typedef`, `uvm_config_wrapper`, is declared for a `uvm_config_db` type-parameterized for `uvm_object_wrapper`.

Configuration review:

- The first argument is the context (mapped to `this`).
- The second argument is the pathname to the UVC sequencer, appended with the run phase in which the sequence should execute. The phase, if omitted, defaults to `run_phase()`.
- Third argument is the field name `default_sequence`.
- Last argument is the sequence type name, using `get_type()` to enable compile-time type checking.

© Dolphin Technology Vietnam Center. All rights reserved

Executing UVC Sequence in Test

```
class mytest extends base_test;
  // macro and constructor
  ...
  uart_tx_seq tx_seq;

  function void build_phase(uvm_phase phase);
    tx_seq = uart_tx_seq::type_id::create("tx_seq", this);
    super.build_phase(phase);
  endfunction : build_phase

  task run_phase(uvm_phase phase);
    assert(tx_seq.randomize());
    tx_seq.start(env.uart_tx_agent.sequencer);
    ...
  endtask : run_phase
  ...
endclass
```

Handle for every sequence to be executed

Create sequence instance

Sequence with `rand` properties must be explicitly randomized

Run `tx_seq` sequence instance on sequencer respectively

Dolphin Confidential Information

As an alternative to setting the default sequence of a sequencer, a test class may execute sequences directly on a sequencer.

The procedure is as follows:

- Declare a handle for the sequence(s) to be executed and create all these handles in the build phase of the test.
- Declare a handle for the UVC sequencer(s) on which the sequences will be executed. These handles are not created, but will be linked directly to the UVC sequencer instances.
- In the connect phase of the test, assign the path name of the UVC sequencer instance to the sequencer handle. The pathname must relative to the test instance and use the handle names (not the instance names) of the components in the path. Wildcards are not allowed in the pathname – we are making a direct reference copy of the sequencer instance, not making a configuration setting.
- If the sequence instance contains rand properties, then the sequence must be explicitly randomized to give values to these properties. If you do not randomize the sequence, rand properties are left at their default value. Randomization could be done once in the build_phase, after the instance creation, or many times in the run_phase, as required.
- In the run phase of the test, execute the sequence instance on the sequencer reference using start.

© Dolphin Technology Vietnam Center. All rights reserved

Ending Simulation

An objection (consensus) mechanism is used to elegantly end the simulation.

- Simulation immediately tries to end the run phase
- A sequence must raise an objection when it starts to hold off the phase end
- A sequence must drop the objection on finishing to allow the phase to end
- After a certain time (drain time) with no further objections raised, the phase ends

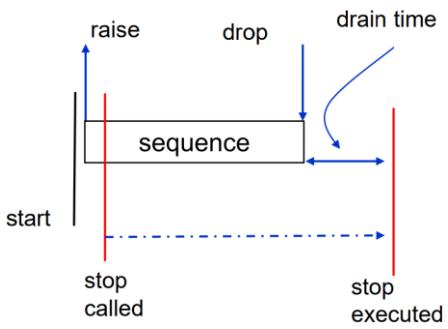
Every run (sub)phase has a built-in objection mechanism.

The run phase ends if no objections were raised or when all objections are removed.

- Expandable, reusable, and flexible

Not all sequences should raise objections

- Not appropriate for background traffic, response generators, or subsequences



Dolphin Confidential Information

The objection mechanism manages the termination of `run_phase` and hence simulation end.

The `run_phase` has a built-in objection mechanism. In addition, every run sub-phase also has its own objection handling.

At the start of every run phase, the objection mechanism tries to end the phase immediately. If a sequence executes in that phase, it must raise an objection to the phase end. Raised objections prevent the phase ending until dropped. Once the sequence has ended, it must drop the objection.

If no objections are raised for a phase, or when the objection count returns to zero, then the phase waits for a certain period of time (called the drain time) in anticipation of another objection being raised. If no new objections are raised during the drain time, the phase ends. The default drain time is zero, i.e., the phase will end immediately when the objection count reaches zero or if no objections are raised.

When all run sub-phases have ended, and the `run_phase` itself has ended, then the simulation progresses to `extract_phase`, `check_phase` and `report_phase`.

Only `run_phase` and it's sub-phases can rise objections as these are the only phases which consume time.

For efficiency and robustness, you should raise the minimum number of objections to maintain the simulation. Therefore not all sequences should handle objections. For example background, response, or nested sequences.

© Dolphin Technology Vietnam Center. All rights reserved

Test Objections

Objections are raised and dropped by methods

```
raise_objection(<object>, <description>, <count>);
drop_objection(<object>, <description>, <count>);
```

- Raise or drop the number of objections for `object` by `count` (default 1)
- `description` is a string used for tracing and debug

In a test, objection methods are called in the run phase (or subphase)

- Using the `phase` method argument

```
class mytest extends base_test;
  ...
  task run_phase(uvm_phase phase);
    phase.raise_objection(this, "mytest");
    assert(tx_seq.randomize());
    tx_seq.start(env.uart_tx_agent.sequencer);
    ...
    phase.drop_objection(this, "mytest");
  endtask : run_phase
  ...
```

Raise an objection for this component for run phase

Drop objection after stimulus sent

Dolphin Confidential Information

Objections are raised and dropped via methods:

```
virtual function void raise_objection (uvm_object obj = null, string
description = "", int count = 1)
```

Raises the objection count for the nominated object (use `this` to specify the calling object) by `count`. The `description` string argument is used for debug.

```
virtual function void drop_objection (uvm_object obj = null, string
description = "", int count = 1)
```

Drops the objection count for the nominated object (use `this` to specify the calling object) by `count`. The `description` string argument is used for debug. Dropping the objection count below 0 is an error.

In a test class, objections are raised and dropped at the beginning and end of the run phase method and called from the `phase` argument of the method.

Objections propagate up the hierarchy to `uvm_top` so that the simulation can keep track of objections raised by every UVC. Once the `uvm_top` objection count for a phase returns to zero (or if no objections are raised for a phase) the phase ends.

© Dolphin Technology Vietnam Center. All rights reserved

17

Register Modeling in UVM



Dolphin Technology Vietnam Center

...to be Silicon IP Experts

© Dolphin Technology Vietnam Center. All rights reserved

Module Objectives

In this module, you

- Apply a simple approach to UVM Register Verification

This module is in four parts:

- Introduction
  - Describe the purpose and aim of register modeling
- Generation
  - Create and test a register reference model
- Integration
  - Add the model to an environment and run built-in tests
- Simulation
  - Create user-defined stimulus

© Dolphin Technology Vietnam Center. All rights reserved



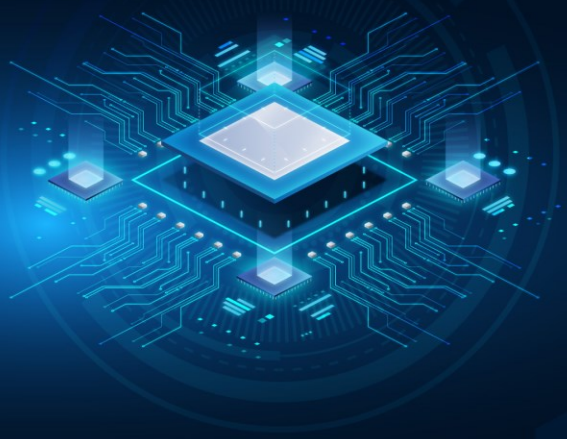
Dolphin Technology Vietnam Center

...to be Silicon IP Experts

©Dolphin Technology Vietnam Center. All rights reserved.
Author: Lam Ngoc Pham
Phone: +84 336 488 047
Email: lampn0@dolphin-vc.com

© Dolphin Technology Vietnam Center. All rights reserved

Dolphin Technology



Thanks for your attention

Dolphin Technology Vietnam Center
2nd Floor, Lilama Building, 124 Minh Khai, Hai Ba Trung Dist., Hanoi
+84 243 624 9784
https://www.dolphin-vc.com
jobs@dolphin-vc.com

© Dolphin Technology Vietnam Center. All rights reserved